

**HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY
SCHOOL OF ELECTRICAL AND ELECTRONIC ENGINEERING**



THESIS

Development of an In-the-Wild Hand Gesture Recognition Model Using Synthetic Multi-View Images

NGUYEN MAI HUONG

Huong.NM224314@sis.hust.edu.vn

Major : Digital Communication and Multimedia Engineering

Thesis advisor : Assoc. Prof. Vu Hai

Signature of advisor

Department : Department of Communication Engineering
Institute : School of Electrical and Electronic Engineering

Hanoi, 6-2026

THESIS ASSIGNMENT

1. Student's information :

Name : Nguyen Mai Huong.

Phone : 0966 215688 Email: Huong.NM224314@sis.hust.edu.vn

Class : ET-E16 02 K67

Affiliation : Hanoi University of Science and Technology - School of Electrical and Electronic Engineering.

Duration : 10/2022 - 07/2026.

2. Thesis title : Development of an In-the-Wild Hand Gesture Recognition Model Using Synthetic Multi-View Images

3. Thesis statement :

This thesis aims to develop a skeleton-based framework for in-the-wild static hand gesture recognition using synthetic multi-view generation. The proposed framework reconstructs plausible 3D hand poses from observed skeletons, generates synthetic multi-view observations through a Blender-based pipeline, and uses training-time multi-view skeleton supervision to improve viewpoint robustness while preserving single-view inference.

4. Declarations/Disclosures :

I declare that this thesis is my own work carried out under the supervision of Prof. Vu Hai. All sources, datasets, tools, and references used in this thesis are properly acknowledged and cited.

Hanoi, June 2026

Author

Nguyen Mai Huong

5. Attestation of thesis advisor:

.....
.....

Hanoi, June 2026

Thesis Advisor

Assoc. Prof. Vu Hai

Abstract

This thesis presents a synthetic multi-view learning framework for skeleton-based hand gesture recognition using a single observed hand pose. Existing hand gesture recognition methods often require large-scale multi-view datasets or multiple synchronized cameras to improve viewpoint robustness. However, collecting such data is expensive and difficult in practical applications. To address this limitation, this work proposes a geometry-driven pipeline that generates synthetic multi-view training samples from a single input skeleton without requiring additional manual annotation.

The proposed framework first estimates a plausible three-dimensional hand pose from a two-dimensional hand skeleton through template fitting and hierarchical geometric refinement. The refined pose is then retargeted to a rigged Blender hand model, allowing realistic synthetic observations to be rendered from multiple virtual viewpoints. Besides RGB images, the rendered poses are projected back into camera-space skeletons, enabling multi-view skeleton supervision without depending on image appearance.

Based on the generated camera-space skeletons, an original-anchored multi-view training framework is introduced. During training, the original skeleton and its synthetic views are processed by a shared GRU encoder. Cross-entropy classification loss and consistency regularization are jointly optimized to encourage viewpoint-invariant feature learning, while inference requires only the original input skeleton.

Experiments are conducted using the CanonicalSet dataset for training and the independent HandinWildSet dataset for external evaluation. The proposed synthetic multi-view training improves raw-skeleton recognition on the external HandinWildSet from $80.86 \pm 2.78\%$ to $86.88 \pm 1.40\%$, while preserving single-skeleton inference. Additional experiments further analyze the influence of template-based refinement and geometry-refined skeleton representations, providing insight into the potential and limitations of the proposed framework.

The results demonstrate that geometry-driven synthetic multi-view supervision is an effective strategy for improving viewpoint robustness in skeleton-based hand gesture recognition while requiring only a single observed hand pose during both data acquisition and inference.

Acknowledgments

First and foremost, I would like to express my deepest gratitude to my supervisor, Prof. Vu Hai, for his guidance, patience, and support throughout this graduation thesis. His comments, suggestions, and encouragement have helped me overcome many difficulties and improve this work step by step. I am truly grateful for the opportunity to learn from him.

I would also like to extend my sincere appreciation to all lecturers of the School of Electrical and Electronic Engineering, Hanoi University of Science and Technology. Their dedication and valuable teachings have provided me with not only academic knowledge but also the mindset and motivation to pursue continuous learning and self-improvement.

My heartfelt thanks go to my friends and classmates of the ET-E16 cohort. Over the past years, we have shared countless lectures, assignments, examinations, and unforgettable moments together. Through long days and sleepless nights of studying, encouraging one another, and growing side by side, they have made my university journey far more meaningful and memorable. I am truly grateful for their companionship, friendship, and all the beautiful memories that we have created together.

Most importantly, I would like to express my deepest gratitude to my beloved family. Their unconditional love, understanding, sacrifices, and support have always been my greatest source of strength. No matter how difficult the journey became, their trust and encouragement gave me the confidence to keep moving forward.

Finally, I would like to thank everyone who has directly or indirectly contributed to this thesis. Although this work still has limitations, it represents my sincere effort and an important milestone in my academic journey. I hope that the knowledge and experience gained during this project will become a valuable foundation for my future path.

Nguyen Mai Huong

Contents

Abstract	i
List of Figures	v
List of Tables	vii
1 INTRODUCTION	1
1.1 Background and motivation	1
1.2 Problem statement	4
1.3 Research objectives	5
1.4 Scope of the thesis	6
1.5 Research methodology	7
1.6 Contributions of the thesis	8
1.7 Thesis organization	8
2 BACKGROUND AND RELATED WORK	9
2.1 Hand gesture recognition	9
2.2 Hand pose estimation	10
2.3 Hand kinematic model	12
2.4 Synthetic data generation	13
2.4.1 Blender hand rig and retargeting	14
2.5 Multi-view learning	14
2.6 RGB-based and multimodal approaches	15
2.7 Related works	16
2.8 Chapter summary	17
3 PROPOSED METHOD	18
3.1 Overview	18
3.2 System input and output	19
3.3 Hand pose representation	22

3.3.1	Twenty-one-joint hand skeleton	22
3.3.2	Kinematic graph structure	24
3.3.3	Palm and fingertip anchor sets	24
3.4	Coordinate systems	25
3.4.1	Observation and template coordinate systems	25
3.4.2	Blender world and rendering camera coordinate systems	26
3.5	Canonical 3D hand template	27
3.5.1	Motivation for using a canonical template	27
3.5.2	Template skeleton representation	28
3.5.3	Template skeleton construction	30
3.6	Weak-perspective projection model	31
3.6.1	From perspective projection to weak-perspective projection	31
3.6.2	Projection of the template skeleton	32
3.7	Global template alignment	33
3.7.1	Rotation matrix parameterization	34
3.7.2	Palm anchor-based alignment	35
3.7.3	Global viewpoint selection	37
3.8	Hierarchical skeleton refinement	39
3.8.1	Motivation for local refinement	40
3.8.2	Pivot-based local transformation	41
3.8.3	Finger refinement groups	43
3.8.4	Local refinement objective	44
3.8.5	Parent-to-child refinement order	45
3.8.6	Hierarchical refinement procedure	45
3.9	Retargeting to the Blender hand rig	47
3.9.1	Skeleton segment direction	48
3.9.2	Bone direction in the Blender rig	48
3.9.3	Quaternion-based direction alignment	49
3.9.4	Pose bone matrix update	50
3.9.5	Joint-to-bone correspondence	51
3.9.6	Retargeting procedure	52
3.10	Synthetic multi-view rendering and camera-space skeleton generation	53
3.10.1	Virtual camera placement	54
3.10.2	Orthographic rendering	55
3.10.3	Rendering configuration	55
3.10.4	Appearance and background variation	56
3.10.5	Camera-space skeleton view generation	57

3.10.6	Generated dataset organization	58
3.10.7	Summary of the rendering stage	58
3.11	Skeleton-based recognition model	59
3.12	Original-anchored multi-view skeleton supervision	60
3.13	Template-based refinement analysis	62
3.14	Overall algorithm	63
3.15	Chapter summary	63
4	EXPERIMENTS AND RESULTS	65
4.1	Experimental setup	65
4.2	Dataset description and preparation	66
4.2.1	Dataset description	66
4.2.2	Skeleton preprocessing	68
4.2.3	Synthetic multi-view generation	69
4.2.4	Dataset usage protocol	70
4.3	Implementation details	70
4.4	Evaluation metrics	71
4.5	Experimental protocols	72
4.6	Quantitative results	73
4.6.1	Confusion matrix analysis	74
4.7	Qualitative results	75
4.7.1	Template fitting	76
4.7.2	Hierarchical refinement	76
4.7.3	Rig retargeting	77
4.7.4	Synthetic multi-view rendering	78
4.7.5	Recognition examples	78
4.8	Discussion	79
4.9	Limitations	80
4.10	Chapter summary	81
	CONCLUSIONS AND FUTURE WORK	82
	Conclusions	82
	Future work	83
	References	84

List of Figures

1.1	Viewpoint variation in static hand gesture recognition	2
1.2	Original seven gesture categories before label merging.	6
3.1	Overall geometry-driven pipeline for skeleton-guided synthetic multi-view hand data generation.	19
3.2	Input and output of the proposed system.	22
3.3	Twenty-one-joint hand skeleton topology used in the proposed method. .	23
3.4	Coordinate systems used in the proposed pipeline: image coordinates, template coordinates, Blender world coordinates, and rendering camera coordinates.	27
3.5	Wrist-centered template normalization. The original template skeleton is translated so that the wrist joint becomes the origin of the template coordinate system.	29
3.6	Canonical 3D hand template and its extracted 21-joint skeleton.	30
3.7	Weak-perspective projection used to align the canonical 3D template skeleton with the observed 2D skeleton.	33
3.8	Example of template fitting before and after hierarchical refinement. The observed 2D skeleton, initial template projection, and refined template projection are visualized for comparison.	41
3.9	Local rotation around a pivot joint. Downstream joints are transformed around the pivot, while the pivot and unrelated joints remain fixed. . . .	42
3.10	Hierarchical finger refinement groups. Each refinement group consists of a pivot joint and its affected downstream joints. Transformations propagate from parent joints toward distal joints following the anatomical finger hierarchy.	43
3.11	Quaternion-based bone direction alignment.	50
3.12	Examples of appearance and background variations.	57
3.13	Architecture of the proposed multi-view skeleton supervision model . . .	62

4.1	Representative RGB samples from the hand gesture datasets	68
4.2	Virtual camera layout used for generating eight synthetic multi-view observations of each hand pose.	69
4.3	Dataset usage and evaluation protocol	70
4.4	Confusion matrices of the raw skeleton baseline and the raw multi-view model on HandinWildSet. Values are normalized by ground-truth class. .	74
4.5	Representative failure cases on HandinWildSet. Errors mainly occur when gestures have similar folded-finger structures, open-palm ambiguity, partial finger visibility, or unclear thumb visibility.	75
4.6	Examples of global template alignment between the observed skeleton and the projected canonical template.	76
4.7	Comparison between the initial fitting result and the refined skeleton projection. Hierarchical refinement improves local finger articulation while preserving the hand topology.	77
4.8	Examples of retargeting fitted skeletons to the Blender hand rig and generating posed hand meshes.	77
4.9	Examples of synthetic multi-view rendering generated from one reconstructed hand pose.	78
4.10	Representative examples of successful recognition and failure cases. Errors mainly occur when gestures have similar finger configurations or when raw skeleton estimation is unstable.	79

List of Tables

3.1	Correspondence between fitted skeleton segments and Blender hand rig bones.	51
3.2	Rendering configuration	55
4.1	Class distribution of the CanonicalSet dataset.	66
4.2	Class distribution of the HandinWildSet external test dataset.	67
4.3	Number of synthetic camera-space skeleton views generated from CanonicalSet.	67
4.4	Summary of datasets and their roles in the experimental protocol.	67
4.5	Main model and training configuration.	71
4.6	Main experimental protocols.	73
4.7	Summary of main experimental results on HandinWildSet.	73

Chapter 1

INTRODUCTION

1.1 Background and motivation

Hand gesture recognition is an important problem in computer vision and human–computer interaction. It allows users to communicate with digital systems through natural hand poses or hand movements, and has been applied in virtual reality, augmented reality, robotics, sign language interpretation, rehabilitation, and intelligent user interfaces [1]. In practical applications, a recognition system should not only classify gestures correctly in controlled environments, but also remain stable when camera viewpoint, lighting condition, hand appearance, and background change.

Recent deep learning approaches have improved the performance of vision-based gesture recognition. Existing methods can generally be divided into two groups: image-based methods and skeleton-based methods. Image-based methods use RGB images as input and learn visual features directly from pixel data. Because RGB images contain hand texture, color, contour, shading, and contextual information, they provide rich cues for classification. However, these cues are also affected by background clutter, illumination changes, skin appearance, occlusion, and viewpoint variation [2]. As a result, image-based models often require large and diverse training sets to generalize well.

Skeleton-based methods represent a hand pose using a set of landmarks corresponding to anatomical joints and fingertips. With practical hand pose estimation frameworks such as MediaPipe Hands [3], twenty-one hand landmarks can be extracted from monocular RGB images in real time. Compared with RGB images, skeletons provide a compact geometric representation and remove most background and appearance information. This property has made skeleton-based learning an effective direction for action and gesture recognition tasks [4]. It is also suitable for hand-in-the-wild settings, where the recording

environment cannot be fully controlled.

Despite these advantages, skeleton-based recognition still faces a major difficulty: viewpoint variation. A gesture observed from the front, from the side, or from an oblique angle may produce different skeleton configurations. Finger self-occlusion, perspective distortion, and palm rotation can change the relative positions of landmarks. At the same time, different gestures may become geometrically similar under unfavorable viewpoints. This issue is especially important for static gestures because their labels often depend on small differences in finger bending, fingertip contact, or finger spacing.

Figure 1.1 illustrates the main viewpoint challenge considered in this thesis. The same hand gesture may generate different observations when viewed from different camera positions, while some different gestures may appear similar under partial occlusion or an unfavorable angle.

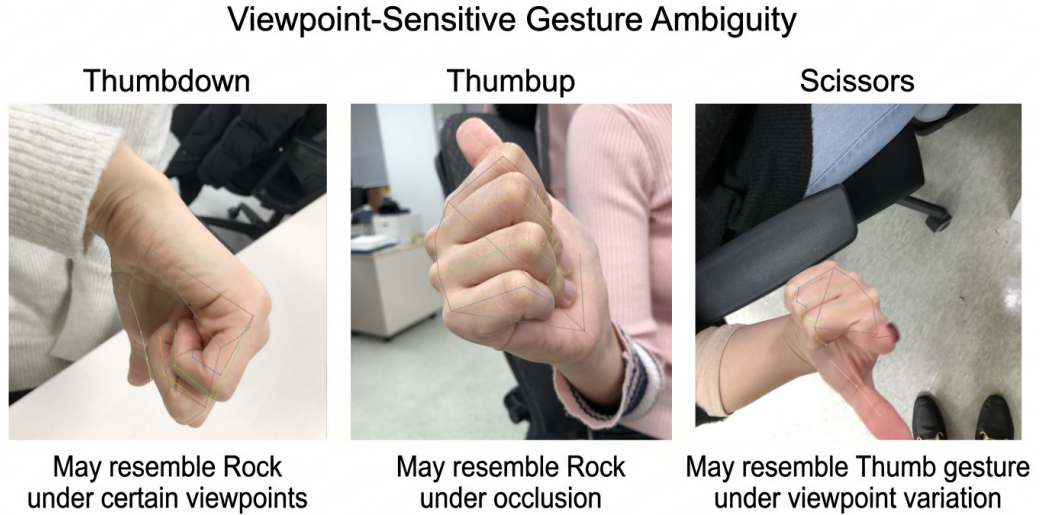


Figure 1.1: Illustration of viewpoint variation in static hand gesture recognition. The same gesture can produce different skeleton observations under different camera viewpoints, while different gestures may become ambiguous when important fingers are occluded.

A direct way to address viewpoint variation is to collect real multi-view datasets. In practice, this solution is expensive and difficult to scale. Multi-view data collection usually requires several synchronized cameras, camera calibration, controlled recording conditions, and additional annotation effort. More importantly, many hand gesture datasets used for testing provide only single-view samples. Therefore, a model that requires multiple camera views during inference is difficult to deploy in realistic settings. A more practical direction is to use multi-view information during training while keeping the inference stage single-view.

Conventional augmentation methods such as image rotation, scaling, cropping, and flipping can increase the amount of training data, but they only modify samples in the image plane. These operations cannot accurately reproduce three-dimensional viewpoint changes, such as changes in palm orientation or finger self-occlusion. Synthetic multi-view generation provides a more suitable alternative, since synthetic observations can increase viewpoint diversity when real multi-view data are difficult to collect [2, 5]. If a hand pose can be represented in three-dimensional space, virtual cameras can be placed around it to generate observations from different viewpoints while preserving the original gesture label.

This thesis follows that direction by using Blender as a controllable geometric simulation environment [6]. A skeleton-driven pipeline is developed to reconstruct plausible hand poses, retarget them to a rigged hand mesh, and observe them from multiple virtual camera viewpoints. The rendered RGB images are mainly used for visualization and qualitative analysis. For recognition, the main training signal comes from the corresponding camera-space skeleton observations. In this way, the framework benefits from viewpoint diversity created by the Blender pipeline while keeping the final recognition model skeleton-based.

The final recognition model is based on 3D hand skeletons. A GRU encoder [7] is used to classify static hand gestures from ordered hand landmarks. During training, each original skeleton is paired with several synthetic camera-space skeleton views generated from virtual camera viewpoints. An original-anchored consistency loss encourages the embeddings of these synthetic views to remain close to the embedding of the original skeleton. During testing, only one observed skeleton is required. This setting is suitable for single-view hand gesture datasets and allows the contribution of synthetic multi-view supervision to be evaluated clearly.

In addition to the main raw-skeleton recognition setting, the thesis also investigates template-based skeleton fitting and refinement. This part reconstructs a more plausible three-dimensional hand configuration from an observed skeleton, transfers the fitted pose to a Blender hand rig, and produces rendered multi-view observations for analysis. The fitted-skeleton experiments are treated as diagnostic studies rather than the main deployable setting. They are used to examine whether geometric refinement can improve recognition and to analyze the mismatch between raw skeletons and fitted skeletons. This analysis is necessary because a refined skeleton may look visually better, but it is not automatically compatible with a model trained or tested on raw landmark data.

1.2 Problem statement

Although recent hand gesture recognition methods have achieved good results, recognizing static gestures under hand-in-the-wild conditions remains challenging. Image-based methods can exploit rich appearance information, but they are sensitive to lighting, background, skin appearance, and occlusion. Skeleton-based methods reduce these visual variations by using landmark coordinates, but the extracted skeletons are still affected by viewpoint changes and pose estimation errors.

The first problem addressed in this thesis is viewpoint robustness, which remains an important issue in skeleton-based recognition under cross-view and cross-dataset settings [8, 9]. A skeleton classifier trained only on the original viewpoint distribution may not generalize well to data captured from different camera positions or recording conditions. Collecting real multi-view data for every target environment is not practical. For this reason, the thesis investigates whether synthetic camera-space skeleton views can be generated and used during training to improve robustness without changing the inference input.

The second problem is the practical use of multi-view information. Multi-view supervision can help the model learn viewpoint-consistent features, but requiring multiple views at test time would make the system less useful in practice. Many hand gesture datasets provide only one observation per sample. Therefore, the recognition framework in this thesis uses synthetic multi-view observations only during training and keeps single-skeleton inference during testing.

The third problem concerns skeleton quality. Raw skeletons extracted from hand pose estimation pipelines may contain unstable joints, distorted fingers, or geometrically implausible configurations. Template-based fitting can produce more regular hand structures, but fitted skeletons and raw skeletons may belong to different representation domains. A model trained on fitted skeletons may perform poorly when it is evaluated on raw skeletons. This raw–fitted domain mismatch must be examined before template refinement can be considered useful for deployment.

These issues directly affect the practical value of a hand gesture recognition system. A model that performs well only on the source data may fail when tested on data collected from another camera setup or environment. Similarly, a model that depends on multiple views during inference is difficult to apply to common single-view datasets. Therefore, this thesis focuses on a training-time multi-view strategy that improves viewpoint robustness while keeping the deployed model simple.

Based on these problems, the central research question of this thesis is formulated as follows:

How can synthetic camera-space skeleton observations generated from a skeleton-driven hand model be used as training-time supervision to improve view-point robustness and cross-dataset generalization for skeleton-based static hand gesture recognition, while preserving single-skeleton inference?

To answer this question, the thesis studies three connected aspects: synthetic multi-view skeleton supervision, template-based skeleton refinement, and evaluation on both source-domain and hand-in-the-wild data.

1.3 Research objectives

The main objective of this thesis is to develop a skeleton-based static hand gesture recognition framework that uses synthetic multi-view generation to improve viewpoint robustness while preserving single-view inference.

The specific objectives are:

1. To construct a structured hand representation based on a twenty-one-joint skeleton topology. This representation is used consistently for recognition, synthetic view generation, template fitting, and result visualization.
2. To build a Blender-based synthetic multi-view generation pipeline. Starting from an observed skeleton, the system reconstructs a plausible 3D hand pose, transfers the pose to a rigged hand mesh, and observes the same pose from several virtual camera viewpoints.
3. To implement a skeleton-based recognition model for static hand gestures. A GRU encoder processes ordered 3D landmarks and produces a compact representation for gesture classification.
4. To evaluate original-anchored synthetic multi-view supervision. Synthetic camera-space skeleton views are used during training to encourage viewpoint-consistent representations, while inference remains based on a single observed skeleton.
5. To analyze the effect of template-based skeleton refinement. The thesis evaluates refined skeletons and raw–refined representation differences as additional diagnostic experiments for understanding the potential and limitations of geometric refinement.

1.4 Scope of the thesis

This thesis focuses on static hand gesture recognition. Dynamic gesture sequences, hand motion trajectories, temporal action recognition, and continuous gesture segmentation are not considered. Each sample is treated as an independent hand pose.

The input of the recognition framework is a twenty-one-joint 3D hand skeleton extracted from monocular RGB hand images. The thesis assumes that hand landmarks are available and does not develop a new hand detector or hand pose estimation network. MediaPipe Hands is used as a practical landmark extraction tool [3]. The main work of the thesis lies in synthetic multi-view generation, skeleton-based representation learning, template-based refinement, and cross-dataset evaluation.

The original gesture categories include *OK*, *Paper*, *Rock*, *Scissors*, *The-Finger*, *Thumb-Up*, and *Thumb-Down*, as illustrated in Figure 1.2.

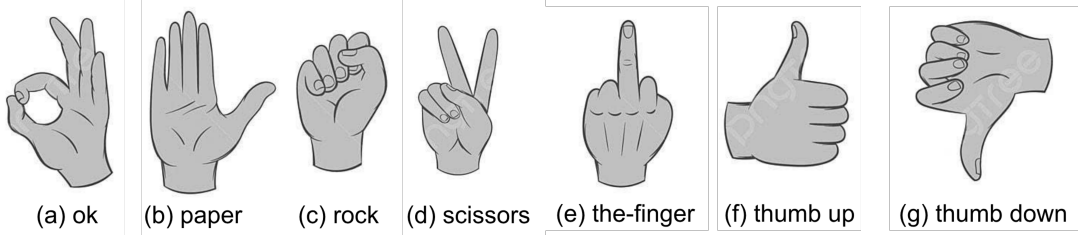


Figure 1.2: Original seven gesture categories before label merging.

In this thesis, *Thumb-Up* and *Thumb-Down* are merged into one *Thumb* class during evaluation. As a result, the final recognition protocol contains six classes. This setting reduces label ambiguity and focuses the evaluation on the main hand-shape categories.

The experiments use CanonicalSet as the source dataset for training, validation, internal testing, and synthetic multi-view generation. HandinWildSet is used as an external hand-in-the-wild test dataset to evaluate generalization under different data conditions. It is not used for training, validation, checkpoint selection, or hyperparameter tuning. This strict separation is important because it allows the experiments to measure whether the model can generalize beyond the source dataset.

The reconstruction and rendering stages are designed to generate plausible hand poses and multi-view visualizations. They are not intended to recover physically exact hand shapes. Since a monocular skeleton does not contain complete physical information about the original hand, the fitted pose should be interpreted as a geometrically consistent approximation rather than a precise reconstruction.

Although synthetic RGB images are generated by the Blender pipeline, RGB-based recognition is not the primary final model in this thesis. Rendered images are used mainly for visualization and qualitative analysis. The quantitative recognition experiments focus on skeleton-based models, synthetic camera-space skeleton supervision, template refinement, and raw-fitted domain mismatch.

1.5 Research methodology

The methodology consists of four main stages: skeleton preparation, synthetic multi-view generation, skeleton-based recognition, and refinement analysis.

First, each hand gesture sample is represented by twenty-one 3D landmarks. The skeleton is normalized by translating the wrist joint to the origin and scaling the hand using a reference palm distance. This step reduces the effect of global position and hand size while preserving the relative configuration of the fingers.

Second, a Blender-based generation pipeline is constructed. A canonical hand template and a rigged hand mesh are used to obtain a plausible 3D hand pose from the input skeleton. Global template fitting estimates the coarse hand orientation, scale, and position. Hierarchical refinement then adjusts local finger articulation. After refinement, the fitted skeleton is retargeted to the Blender hand rig, and virtual cameras are placed around the posed hand. The rendered images are used for visualization, while the generated camera-space skeleton observations are used for multi-view supervision.

Third, a GRU-based skeleton classifier is trained. The original skeleton and its synthetic camera-space views are processed by a shared encoder. Training uses a cross-entropy classification loss and an original-anchored consistency loss. The classification loss supervises gesture prediction, while the consistency term encourages synthetic view embeddings to stay close to the original skeleton embedding. At inference time, only the original skeleton is used.

Finally, the role of template-based refinement is studied through additional diagnostic experiments. Raw skeleton training, fitted skeleton training, synthetic multi-view training, oracle fitted evaluation, and label-free template selection are compared. This analysis helps clarify when refinement is useful and why a visible improvement in skeleton geometry does not always translate directly into better deployable recognition performance.

1.6 Contributions of the thesis

The main contributions of this thesis are summarized as follows:

1. A geometry-driven pipeline is developed for generating synthetic multi-view hand observations from a single input skeleton. The pipeline combines 3D template fitting, hierarchical refinement, Blender rig retargeting, and virtual-camera-based multi-view generation in one workflow.
2. A skeleton-based multi-view supervision strategy is implemented for static hand gesture recognition. Synthetic camera-space skeletons are used during training through an original-anchored consistency objective, while the final model still requires only one skeleton at inference time.
3. A cross-dataset experimental evaluation is conducted using CanonicalSet for model development and HandinWildSet for external generalization analysis. The experiments analyze raw skeleton learning, synthetic multi-view supervision, and geometry-refined skeletons, thereby showing both the benefits and the limitations of the proposed framework.

1.7 Thesis organization

The remainder of this thesis is organized as follows.

Chapter 2 presents the necessary background and related work, including hand gesture recognition, hand pose estimation, hand skeleton representation, synthetic data generation, multi-view learning, and skeleton-based recognition.

Chapter 3 describes the proposed framework. It covers the hand pose representation, canonical hand template, global template fitting, hierarchical skeleton refinement, Blender rig retargeting, synthetic multi-view generation, GRU-based recognition model, and original-anchored multi-view consistency learning.

Chapter 4 presents the experimental setup and results. It describes the dataset protocol, implementation details, evaluation metrics, baseline configurations, raw skeleton experiments, synthetic multi-view experiments, template refinement analysis, label-free template selection, and qualitative results.

Finally, the final chapter summarizes the main findings, discusses the limitations of the current work, and suggests possible directions for future development.

Chapter 2

BACKGROUND AND RELATED WORK

2.1 Hand gesture recognition

Hand gesture recognition aims to identify the semantic meaning of a hand pose or hand movement from visual observations. It is widely used in human–computer interaction, virtual reality, augmented reality, robotics, sign language interpretation, rehabilitation, and smart environments [1]. Based on the input representation, existing methods can be broadly divided into image-based methods and skeleton-based methods.

Image-based methods use RGB images as the input representation. Early approaches relied on handcrafted features such as Scale-Invariant Feature Transform, Histogram of Oriented Gradients, and Local Binary Patterns, which were then combined with conventional classifiers such as Support Vector Machines or Random Forests. With the development of deep learning, convolutional neural networks have become widely used for visual recognition tasks [10]. These models learn discriminative visual features directly from data and have also been applied to hand gesture recognition.

RGB images preserve rich visual information, including hand contour, skin appearance, shading, texture, and background context. These cues are useful when gestures are visually distinct. However, the same cues also make RGB-based models sensitive to environmental changes. Lighting, background clutter, skin tone, camera distance, and viewpoint variation may affect the learned representation. As a result, image-based methods usually require large and diverse datasets to generalize well under hand-in-the-wild conditions.

Skeleton-based methods use hand landmarks as the input representation. Instead of processing raw pixels, these methods operate on joint coordinates corresponding to the

wrist, finger joints, and fingertips. A skeleton representation removes most background and appearance information and keeps the geometric structure of the hand. This makes it compact, efficient, and less sensitive to lighting or texture changes. Skeleton-based learning has therefore become an important direction in action and gesture recognition [4].

For hand gesture recognition, the skeleton representation is especially useful because many static gestures are defined by finger articulation and fingertip arrangement. A twenty-one-joint hand skeleton can describe the main structure of the palm and five fingers while remaining computationally lightweight. Learning models such as recurrent neural networks, graph-based networks, and transformer-based models can then be used to extract discriminative features from these joint coordinates.

Recurrent models, including gated recurrent units, are suitable when a skeleton is represented as an ordered sequence of joints. Although the gestures considered in this thesis are static rather than temporal sequences, the hand landmarks can still be arranged according to the hand topology and processed sequentially. A GRU encoder can aggregate joint-level information from the wrist to the fingertips and produce a compact feature vector for classification [7]. This design provides a simple and efficient baseline for skeleton-based static gesture recognition.

Despite these advantages, skeleton-based methods still face several limitations. Hand landmark coordinates depend on the camera viewpoint, so the same gesture may appear different when observed from another direction. Self-occlusion and pose estimation errors may also produce unstable or ambiguous joint configurations. These issues are particularly important for static gestures, where small differences in finger bending or fingertip contact may determine the class label. Viewpoint robustness therefore remains a key challenge for skeleton-based hand gesture recognition.

In this thesis, skeleton-based recognition is used as the main recognition direction. RGB images and rendered views are not used as the final recognition backbone. Instead, they support synthetic multi-view generation, qualitative visualization, and analysis of the reconstructed hand pose. This choice keeps the recognition pipeline compact while allowing the proposed method to exploit viewpoint diversity through synthetic camera-space skeleton supervision.

2.2 Hand pose estimation

Hand pose estimation aims to locate anatomical hand landmarks from visual input. Given an RGB or depth image, a hand pose estimation method predicts the positions of joints such as the wrist, finger bases, intermediate finger joints, and fingertips [11]. The estimated

landmarks are commonly used as an intermediate representation for gesture recognition, sign language understanding, virtual reality interaction, and hand motion analysis.

Hand pose estimation can be formulated as either a two-dimensional or a three-dimensional problem. Two-dimensional methods estimate landmark coordinates in the image plane. Three-dimensional methods attempt to recover the spatial configuration of the hand in 3D space. Although 3D landmarks provide richer geometric information, estimating them from monocular RGB images is difficult because depth is ambiguous and fingers often occlude each other.

Early hand pose estimation methods relied on handcrafted features, geometric priors, and model-based optimization. These methods often required careful initialization and could fail under occlusion or cluttered backgrounds. Deep learning approaches have improved this problem by learning landmark predictors directly from data. Common designs include heatmap-based methods, which estimate probability maps for each joint, and regression-based methods, which predict landmark coordinates directly. Recent survey work shows that efficient annotation, learning strategy, and dataset diversity remain important issues for 3D hand pose estimation [11].

MediaPipe Hands is particularly relevant to this thesis because it provides a practical real-time hand landmark extraction pipeline from monocular RGB images [3]. The framework first detects the palm region and then estimates twenty-one hand landmarks. These landmarks follow a fixed topology consisting of the wrist and five finger chains. This output format is convenient for skeleton-based gesture recognition because it provides a consistent joint order across samples.

MediaPipe Hands produces image-space coordinates and normalized three-dimensional landmark coordinates. The normalized 3D coordinates include relative depth information, but they should not be interpreted as physically exact hand reconstructions. They are designed primarily for landmark tracking and pose representation. Consequently, when these coordinates are used for recognition, they may still contain viewpoint-dependent distortion, noisy fingertip positions, or geometrically implausible finger configurations.

Many gesture recognition systems treat the estimated landmarks as the final input representation. This thesis uses them in two roles. First, the landmarks are used directly as the input to the skeleton-based recognition model. Second, they serve as a geometric starting point for synthetic multi-view generation and template-based refinement. This creates a connection between hand pose estimation, skeleton representation learning, and Blender-based viewpoint simulation.

2.3 Hand kinematic model

The human hand is a highly articulated structure consisting of multiple joints connected by rigid bones. From the perspective of computer vision and computer graphics, the hand can be represented as a kinematic chain, where joints define pivot locations and bones describe the anatomical connections between adjacent joints. Such a representation preserves the structural relationships of the hand and provides a suitable foundation for pose estimation, gesture recognition, retargeting, and synthetic data generation.

In this thesis, the hand is represented using the standard twenty-one-joint topology adopted by MediaPipe Hands and many public hand pose datasets [3]. This representation consists of one wrist joint and twenty finger joints distributed across the thumb, index, middle, ring, and little fingers. Compared with dense mesh representations, the twenty-one-joint topology is compact while still preserving the essential articulation of the hand.

The wrist is defined as the root joint of the kinematic structure. Each finger forms an independent parent–child chain extending from the palm. Specifically, joints 1–4 correspond to the thumb, joints 5–8 to the index finger, joints 9–12 to the middle finger, joints 13–16 to the ring finger, and joints 17–20 to the little finger. This hierarchical organization naturally models the articulation of the human hand because the motion of each child joint depends on the transformation of its parent.

The hand skeleton can be represented as a graph

$$\mathcal{G} = (\mathcal{V}, \mathcal{E}), \quad (2.1)$$

where $\mathcal{V}$ denotes the set of joints and $\mathcal{E}$ denotes the bone connections between adjacent joints. Each joint is associated with a three-dimensional coordinate

$$\mathbf{X}_i = [x_i, y_i, z_i]^T, \quad (2.2)$$

where $i = 0, \dots, 20$. The complete hand skeleton is therefore expressed as

$$\mathcal{X} = \{\mathbf{X}_0, \mathbf{X}_1, \dots, \mathbf{X}_{20}\}. \quad (2.3)$$

Compared with RGB images, the skeleton provides a compact geometric representation that focuses on the spatial arrangement of the hand. It removes most appearance-related information such as texture, color, illumination, and background, allowing the recognition model to concentrate on gesture geometry. However, the observed landmark coordinates still depend on the camera viewpoint and may contain estimation noise, par-

ticularly around the fingertips.

The hierarchical hand topology is also essential for the proposed reconstruction framework. Since each finger forms a parent–child chain, local transformations can be propagated from proximal joints to distal joints while preserving anatomical continuity. This principle is widely used in articulated character animation and inverse kinematics [12], and forms the basis of the template fitting and hierarchical refinement procedures described in Chapter 3.

Therefore, the twenty-one-joint hand topology plays two complementary roles in this thesis. It serves as the input representation for skeleton-based gesture recognition and simultaneously provides the geometric structure required for template fitting, rig retargeting, synthetic multi-view generation, and viewpoint analysis.

2.4 Synthetic data generation

Deep learning models usually require diverse training data to generalize well. However, collecting hand gesture data under many viewpoints, lighting conditions, backgrounds, and hand appearances is expensive and time-consuming. Annotation further increases the cost, especially when three-dimensional landmarks are required. These limitations motivate synthetic data generation.

Synthetic data generation produces artificial samples using simulation or computer graphics and has been widely used in hand pose estimation and hand tracking to reduce annotation cost and improve viewpoint diversity. Compared with manually collected datasets, synthetic data can be generated under controlled conditions. Camera viewpoint, object pose, lighting, material, and background can be adjusted systematically, while labels and geometric information are obtained automatically from the generation process.

Traditional data augmentation methods, including image rotation, scaling, flipping, cropping, and color perturbation, modify images only in the image plane. Although useful, they cannot accurately reproduce three-dimensional viewpoint variation, palm orientation changes, finger self-occlusion, or visibility changes among fingers. A more suitable solution is to generate observations in 3D space and render or project them from virtual cameras. This strategy preserves the gesture label while introducing viewpoint-dependent changes that are difficult to reproduce using image-space augmentation.

Blender is an open-source three-dimensional creation suite supporting modeling, rigging, animation, rendering, and Python scripting [6]. Its flexibility makes it suitable for controlled synthetic data generation. A rigged hand model can be posed automatically, and

virtual cameras can be placed at different viewpoints to generate synthetic observations.

2.4.1 Blender hand rig and retargeting

In the proposed framework, Blender is used not only as a rendering engine but also as a geometric simulation environment. The hand model employed in this thesis is a rigged mesh consisting of two components: a surface mesh representing the visible hand geometry and an internal armature representing the articulated skeletal structure. The armature contains a hierarchy of bones corresponding to the palm and finger segments. During posing, the mesh is deformed through bone transformations rather than by directly modifying mesh vertices.

The fitted hand skeleton and the Blender armature are different representations. The fitted skeleton stores joint coordinates, whereas the armature is controlled by bone rotations. Therefore, retargeting is needed to transfer the reconstructed pose to the rigged hand model. In this thesis, retargeting is performed by aligning each armature bone direction with the corresponding fitted skeleton segment. This preserves the original hand mesh while reproducing the target gesture pose. The complete procedure is described in Chapter 3.

After retargeting, the posed hand model can be observed from multiple virtual camera viewpoints. The rendered images are used for qualitative visualization, while the same camera setup is used to generate camera-space skeleton views for training.

Overall, Blender provides the controllable 3D environment required by the proposed synthetic multi-view generation pipeline.

2.5 Multi-view learning

Multi-view learning aims to use multiple observations or representations of the same sample to improve learning performance. In computer vision, different views may correspond to different camera positions, sensors, modalities, or transformed versions of the same object. The key assumption is that these views share the same semantic label while containing different view-dependent characteristics.

In hand gesture recognition, multi-view learning is important because the appearance and skeleton configuration of a hand gesture can change consistently with camera viewpoint. A gesture that is clear from a frontal view may become ambiguous from a side view due to palm rotation, finger overlap, or self-occlusion. Learning from multiple viewpoints can therefore help a model focus on gesture-related structure rather than viewpoint-

specific patterns. Traditional multi-view learning methods include feature fusion and correlation-based learning. In feature fusion, features extracted from different views are combined before classification. Correlation-based methods, such as Canonical Correlation Analysis and multi-view discriminant analysis, attempt to learn view-consistent representations across different observations [13, 14]. These methods are useful when paired or synchronized multi-view data are available.

However, real multi-view data are often difficult to obtain. Multi-camera capture requires camera synchronization, calibration, and controlled data collection. In addition, many practical hand gesture datasets contain only a single observation for each sample. For this reason, a model that requires multiple views during inference is less practical for hand-in-the-wild recognition.

A more suitable strategy is to use multi-view information only during training. Synthetic views can be generated from the training data, and the model can be encouraged to produce stable representations across these views. During inference, the model receives only a single original skeleton. This setting avoids the need for multi-camera input at deployment while still allowing the model to learn from viewpoint variation.

This thesis adopts such a training-time multi-view strategy. Synthetic camera-space skeletons are generated from the original hand skeleton using the Blender-based pipeline. A shared GRU encoder processes both the original skeleton and the synthetic views. The model is trained with a classification loss and an original-anchored consistency loss, which encourages synthetic view embeddings to remain close to the original skeleton embedding. This design is intended to improve viewpoint robustness while preserving single-skeleton inference.

2.6 RGB-based and multimodal approaches

RGB-based and multimodal approaches are related to this thesis because the proposed pipeline generates synthetic RGB images during reconstruction and rendering. RGB images contain hand contour, texture, skin tone, shading, and occlusion cues. These cues can complement skeleton information, especially when landmarks are noisy or incomplete.

Multimodal learning combines information from different sources, such as RGB images, depth maps, infrared images, hand skeletons, or sensor signals. In hand gesture recognition, RGB and skeleton data can provide complementary information: skeletons describe joint geometry, while RGB images preserve visual appearance and contour information.

However, RGB-based or multimodal recognition depends strongly on the realism and diversity of image data. If synthetic images do not match real-world hand appearances, an RGB branch may not generalize well. For this reason, RGB-based recognition is not the primary final model in this thesis. The generated RGB images are used mainly for qualitative visualization, while the quantitative analysis focuses on skeleton-based learning and synthetic camera-space skeleton supervision.

2.7 Related works

The studies most closely related to this thesis can be grouped into three directions: skeleton-based hand gesture recognition, synthetic data generation, and multi-view representation learning.

Skeleton-based methods use hand landmarks as the main input representation. They are compact and less affected by background or lighting than RGB-based methods. Recurrent models such as GRU can process ordered landmarks, while graph-based and transformer-based models can explicitly model joint relationships [7]. However, skeleton coordinates remain sensitive to camera viewpoint and landmark estimation errors.

Synthetic data generation has been used to increase data diversity and reduce annotation cost. Rendering-based methods can generate samples from different viewpoints while preserving labels. Blender-based augmentation has also been applied to hand gesture recognition under unconstrained conditions [5]. In contrast to RGB-focused augmentation, this thesis uses Blender mainly for skeleton-driven viewpoint simulation and camera-space skeleton generation.

Multi-view learning aims to learn stable representations from different observations of the same sample. Classical methods rely on paired multi-view data or feature correlation across views [13, 14], while recent methods often use consistency or contrastive objectives [15]. However, real multi-view hand gesture data are difficult to collect, and requiring multiple views during inference is not practical for common single-view datasets.

The gap addressed in this thesis is the lack of a practical framework that combines skeleton-based recognition, synthetic multi-view generation, template-based refinement analysis, and single-skeleton inference. The proposed method uses synthetic camera-space skeleton views only during training, while keeping inference simple and deployable.

2.8 Chapter summary

This chapter reviewed the background and related work for the proposed framework. It discussed hand gesture recognition, hand pose estimation, the 21-joint hand skeleton, synthetic data generation, Blender rigging and retargeting, multi-view learning, and RGB-based or multimodal approaches.

The review shows that skeleton-based methods are compact and efficient, but they remain sensitive to viewpoint changes and landmark noise. Real multi-view data are expensive to collect and are not always available in practical hand gesture datasets. Synthetic generation provides a practical way to introduce viewpoint diversity when a hand pose can be reconstructed and observed from virtual cameras.

These observations motivate the framework presented in the next chapter, which combines template-based skeleton fitting, hierarchical refinement, Blender retargeting, synthetic camera-space skeleton generation, and original-anchored multi-view supervision.

Chapter 3

PROPOSED METHOD

3.1 Overview

This chapter presents the proposed geometry-driven framework for in-the-wild hand gesture recognition. Instead of relying only on two-dimensional image transformations such as rotation, cropping, or scaling, the method reconstructs a plausible 3D hand configuration from an observed skeleton. The reconstructed pose is transferred to a rigged hand mesh in Blender and observed from multiple virtual viewpoints.

The generated RGB images are used for visualization, while the corresponding camera-space skeletons are used for training. The final recognition model remains skeleton-based and requires only one skeleton during inference.

The proposed pipeline starts from a sparse hand pose representation. Each input sample is treated as a static hand pose represented by a 21-joint hand skeleton. The system does not reconstruct a continuous temporal sequence. Instead, it estimates a plausible 3D hand pose for each static sample and generates multiple synthetic observations from that pose. This design is appropriate for static gesture recognition, where the identity of a gesture is mainly determined by the spatial arrangement of the palm, fingers, and fingertips.

The central idea of the method is to combine pose-specific information from the observed skeleton with the geometric structure of a canonical 3D hand template. The observed skeleton provides the target gesture configuration, while the 3D template provides anatomical consistency [16], fixed hand proportions, and compatibility with a rigged hand mesh in Blender. By fitting the 3D template skeleton to the observed skeleton, the system obtains a structured 3D representation that can be transferred to the hand rig and rendered from different viewpoints.

The complete pipeline consists of three stages. First, an observed skeleton is fitted to a canonical 3D hand template using weak-perspective alignment and hierarchical refinement. Second, the fitted skeleton is retargeted to a rigged Blender hand mesh and observed from multiple virtual cameras. Third, the generated camera-space skeleton views are used as training-time supervision for a skeleton-based gesture classifier. During inference, the model uses only one skeleton input.

Figure 3.1 illustrates the overall geometry-driven pipeline.

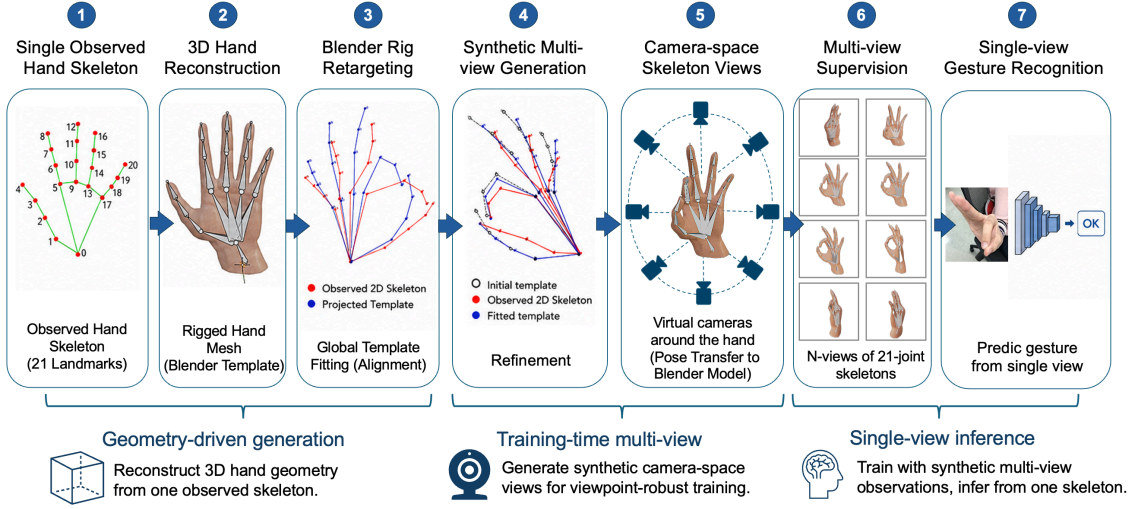


Figure 3.1: Overall geometry-driven pipeline for skeleton-guided synthetic multi-view hand data generation.

3.2 System input and output

The overall geometric generation pipeline can be summarized as

$$\mathcal{U} \rightarrow \bar{\mathcal{X}} \rightarrow \mathcal{X}^{fit} \rightarrow \mathcal{M}^{pose} \rightarrow (\mathcal{R}, \mathcal{V}), \quad (3.1)$$

where $\mathcal{U}$ denotes the observed hand skeleton used for template fitting, $\bar{\mathcal{X}}$ denotes the wrist-centered canonical 3D template skeleton, $\mathcal{X}^{fit}$ denotes the fitted and refined 3D skeleton, $\mathcal{M}^{pose}$ denotes the posed hand mesh after rig retargeting, $\mathcal{R}$ denotes the set of rendered multi-view RGB images, and $\mathcal{V}$ denotes the set of synthetic camera-space skeleton views used for multi-view skeleton supervision.

The recognition pipeline can be summarized as

$$(\mathbf{S}, \mathbf{S}^{(1)}, \dots, \mathbf{S}^{(K)}) \rightarrow (\mathbf{f}_0, \mathbf{f}_1, \dots, \mathbf{f}_K) \rightarrow \mathcal{L}_{cls} + \lambda \mathcal{L}_{con} \rightarrow \hat{y}, \quad (3.2)$$

where $\mathbf{S}$ denotes the normalized observed 3D skeleton, $\mathbf{S}^{(k)}$ denotes the k -th synthetic camera-space skeleton view, $\mathbf{f}_0$ denotes the feature of the original skeleton, $\mathbf{f}_k$ denotes the feature of the k -th synthetic view, $\mathcal{L}_{cls}$ denotes the classification loss, $\mathcal{L}_{con}$ denotes the original-anchored consistency loss, and $\hat{y}$ denotes the predicted gesture label.

The proposed system uses two main representation domains: skeletons and rendered images. The input is a static hand pose represented by landmarks. The intermediate outputs are a fitted 3D skeleton and a posed hand mesh. The final generated outputs are rendered RGB images for visualization and camera-space skeleton views for multi-view supervision.

The recognition input is a 3D hand skeleton:

$$\mathbf{S} = \{\mathbf{p}_0, \mathbf{p}_1, \dots, \mathbf{p}_{20}\}, \quad \mathbf{p}_i = \begin{bmatrix} x_i \\ y_i \\ z_i \end{bmatrix} \in \mathbb{R}^3. \quad (3.3)$$

Here, $\mathbf{p}_i$ denotes the 3D coordinate of the i -th hand landmark. In the recognition experiments, this representation is used in the raw-skeleton setting. A fitted skeleton representation is also evaluated to analyze the effect of template-based refinement. Before being fed into the recognition model, the skeleton is normalized by translating the wrist joint to the origin and scaling the hand according to a reference palm distance.

For template fitting, the observed image-plane skeleton is represented as

$$\mathcal{U} = \{\mathbf{u}_0, \mathbf{u}_1, \dots, \mathbf{u}_{20}\}, \quad \mathbf{u}_i = \begin{bmatrix} u_i \\ v_i \end{bmatrix} \in \mathbb{R}^2. \quad (3.4)$$

Here, $\mathbf{u}_i$ denotes the image-plane coordinate of the i -th hand landmark. The coordinate u_i represents the horizontal image coordinate, while v_i represents the vertical image coordinate. The coordinates may be represented either in pixel units or in normalized image coordinates, as long as all joints of one sample follow the same coordinate convention.

In addition to the observed skeleton, the system uses a canonical 3D hand template. The template consists of a 3D skeleton and a rigged hand mesh. The template skeleton is used during geometric fitting, while the rigged mesh is used during retargeting and rendering. The template does not correspond to a specific input sample. Instead, it provides a reusable geometric prior that can be adapted to different observed poses.

For each input sample, the system produces three major outputs. The first output is a

fitted 3D skeleton:

$$\mathcal{X}^{fit} = \left\{ \mathbf{X}_0^{fit}, \mathbf{X}_1^{fit}, \dots, \mathbf{X}_{20}^{fit} \right\}, \quad \mathbf{X}_i^{fit} \in \mathbb{R}^3. \quad (3.5)$$

This fitted skeleton is an intermediate geometric representation. It stores the reconstructed 3D configuration of the hand after global template alignment and hierarchical skeleton refinement. It provides the pose information required to drive the Blender hand rig and is also used to analyze the effect of template-based refinement.

The second output is a set of rendered RGB images:

$$\mathcal{R} = \{I_1, I_2, \dots, I_K\},$$

where K is the number of rendered views for one input pose. Each image I_k corresponds to the same underlying gesture but is generated under a different camera viewpoint or rendering configuration. Since the rendering process modifies viewpoint and visual appearance while preserving the fitted hand pose, the generated images inherit the gesture label of the input skeleton. In the final recognition setting of this thesis, these RGB images are mainly used for visualization and qualitative validation of the reconstruction pipeline.

The third output is a set of synthetic camera-space skeletons:

$$\mathcal{V} = \{\mathbf{S}^{(1)}, \mathbf{S}^{(2)}, \dots, \mathbf{S}^{(K)}\},$$

where $\mathbf{S}^{(k)}$ denotes the skeleton observed from the k -th virtual camera coordinate system. These synthetic views are used during training to provide viewpoint supervision for the skeleton-based recognition model.

The complete input-output relation of the geometric generation stage can be written as

$$F : (\mathcal{U}, \bar{\mathcal{X}}, \mathcal{M}) \mapsto (\mathcal{X}^{fit}, \mathcal{M}^{pose}, \mathcal{R}, \mathcal{V}), \quad (3.6)$$

where F denotes the complete generation pipeline and $\mathcal{M}$ denotes the rigged hand mesh. This formulation emphasizes that the method is not only a rendering process. It first constructs an intermediate 3D pose representation and then uses that representation to generate synthetic observations. In the experimental implementation, each sample is associated with a gesture label and a 21-joint skeleton representation. The dataset-specific split, class distribution, and evaluation protocol are described in Chapter 4. This chapter focuses on the representation and processing pipeline that can be applied to each skeleton sample.

Throughout this chapter, $\mathcal{U}$ denotes the 2D observed skeleton used for fitting, $\mathbf{S}$ denotes

the normalized 3D skeleton used for recognition, and $\mathcal{X}^{fit}$ denotes the fitted 3D template skeleton used for retargeting and rendering.

The input-output structure of the proposed system is illustrated in Figure 3.2.

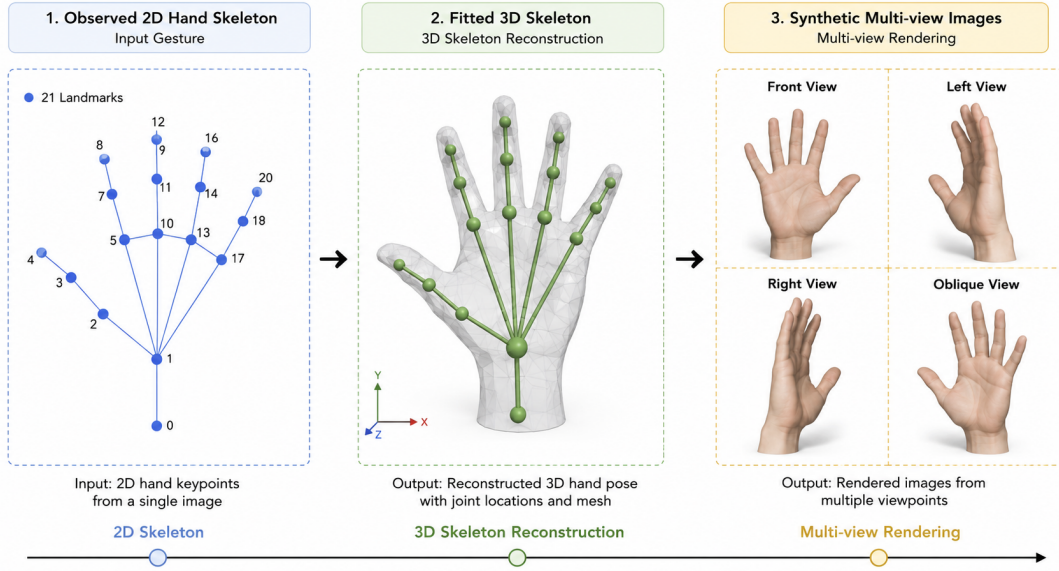


Figure 3.2: Input and output of the proposed system.

3.3 Hand pose representation

3.3.1 Twenty-one-joint hand skeleton

A hand pose is represented using a 21-joint skeleton. This representation provides a compact but expressive description of the hand. Although the real hand has a complex surface geometry, the main pose information can be represented using a limited number of anatomical landmarks located at the wrist, finger bases, intermediate finger joints, and fingertips.

The skeleton follows the common 21-joint topology used in many hand landmark representations. The wrist joint is assigned index 0. The remaining joints are distributed across the thumb, index, middle, ring, and pinky fingers. The five finger chains are defined as

$$\begin{aligned}
 \text{Thumb: } & 0 \rightarrow 1 \rightarrow 2 \rightarrow 3 \rightarrow 4, \\
 \text{Index: } & 0 \rightarrow 5 \rightarrow 6 \rightarrow 7 \rightarrow 8, \\
 \text{Middle: } & 0 \rightarrow 9 \rightarrow 10 \rightarrow 11 \rightarrow 12, \\
 \text{Ring: } & 0 \rightarrow 13 \rightarrow 14 \rightarrow 15 \rightarrow 16, \\
 \text{Pinky: } & 0 \rightarrow 17 \rightarrow 18 \rightarrow 19 \rightarrow 20.
 \end{aligned}$$

This topology is important because the proposed method treats the hand as a structured kinematic system, not as an unordered set of points. A transformation applied to a parent joint affects its downstream joints. This parent–child relationship is used in hierarchical refinement and Blender rig retargeting.

The 21-joint skeleton provides a balance between computational efficiency and geometric expressiveness. It is compact enough for efficient fitting and recognition, but it still captures the main spatial relationships required for static hand gesture recognition. In particular, fingertip locations are important for distinguishing gestures, while palm joints provide information about the global hand structure and orientation.

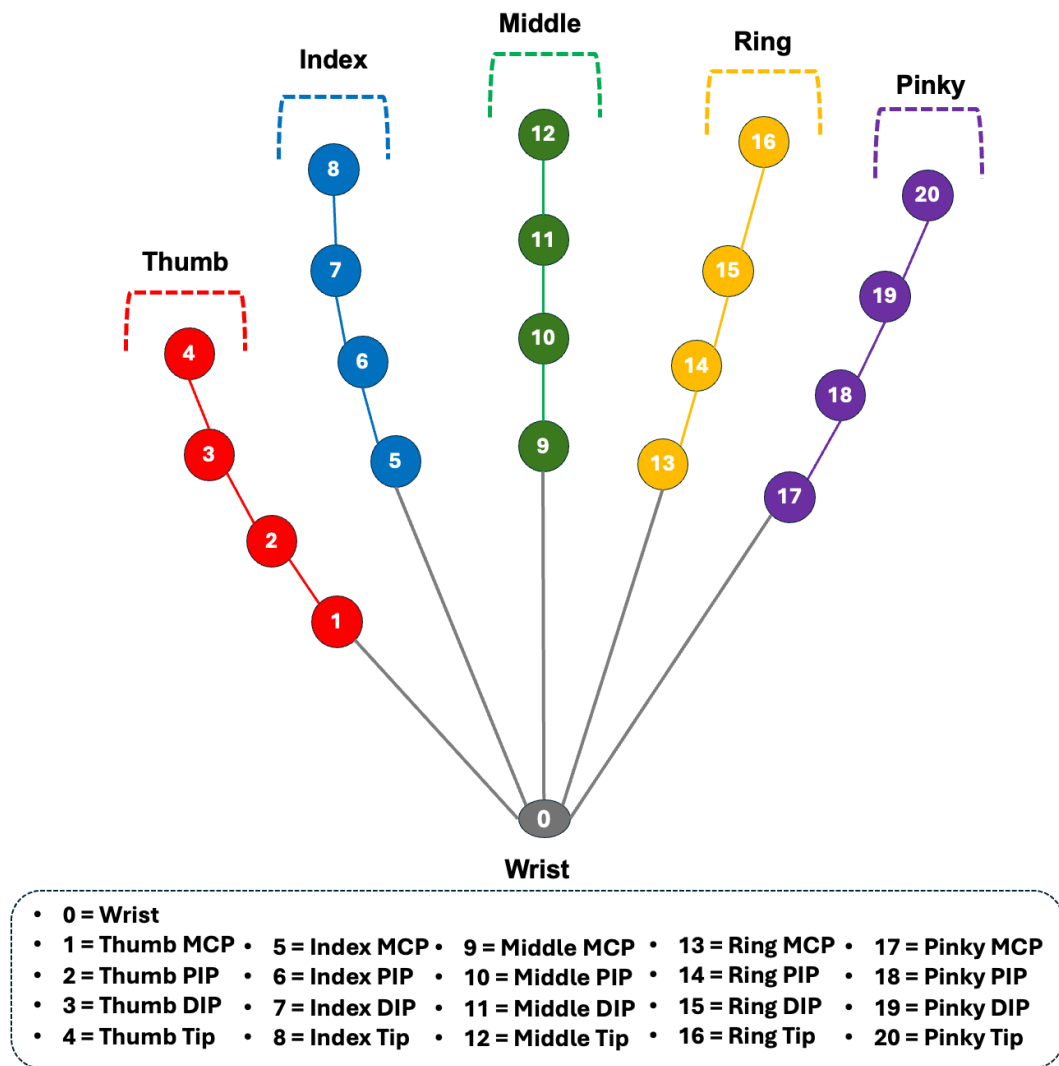


Figure 3.3: Twenty-one-joint hand skeleton topology used in the proposed method.

3.3.2 Kinematic graph structure

The hand skeleton can be represented as a tree-structured graph:

$$\mathcal{G} = (\mathcal{V}_{joint}, \mathcal{E}),$$

where $\mathcal{V}_{joint}$ is the set of joints and $\mathcal{E}$ is the set of bone connections. The vertex set is

$$\mathcal{V}_{joint} = \{0, 1, \dots, 20\}.$$

The edge set contains the parent-child connections of the five fingers:

$$\begin{aligned} \mathcal{E} = \{ & (0, 1), (1, 2), (2, 3), (3, 4), \\ & (0, 5), (5, 6), (6, 7), (7, 8), \\ & (0, 9), (9, 10), (10, 11), (11, 12), \\ & (0, 13), (13, 14), (14, 15), (15, 16), \\ & (0, 17), (17, 18), (18, 19), (19, 20) \}. \end{aligned}$$

This graph representation is used to define finger chains, downstream joints, and bone segments. During skeleton refinement, a local transformation is applied to a selected pivot joint and its downstream joints. During retargeting, each edge in $\mathcal{E}$ corresponds to a bone direction that must be transferred to the Blender hand rig.

The graph formulation also prevents physically implausible transformations. If joints were refined independently, the resulting skeleton could contain disconnected segments or inconsistent finger chains. By preserving the graph topology, the proposed method maintains the anatomical continuity of the hand while still allowing local pose adaptation.

3.3.3 Palm and fingertip anchor sets

Although all 21 joints are used to describe the hand pose, different joint groups play different roles during fitting. Palm joints provide a relatively stable structure for global alignment, while fingertips are highly informative for gesture identity. Therefore, the proposed method defines three joint sets.

The complete joint set is $\mathcal{I}_{all} = \{0, 1, \dots, 20\}$. The fingertip set is $\mathcal{I}_{tip} = \{4, 8, 12, 16, 20\}$. The palm anchor set is $\mathcal{I}_{palm} = \{0, 5, 9, 13, 17\}$.

The set $\mathcal{I}_{palm}$ contains the wrist and the base joints of the four non-thumb fingers. These joints form a stable palm structure and are used as anchors during global alignment. The set $\mathcal{I}_{tip}$ contains the five fingertips and is used to emphasize gesture-defining details

during fitting.

The separation of these joint groups is necessary because a single uniform error over all joints may not reflect the visual importance of different hand regions. For example, in an OK gesture, the spatial relationship between the thumb tip and the index fingertip is critical. A fitted skeleton may have a small average joint error but still appear incorrect if these fingertips are poorly aligned. Conversely, focusing only on fingertips may distort the palm. Therefore, the fitting objective later combines all-joint consistency, fingertip accuracy, and palm stability.

3.4 Coordinate systems

The proposed method uses several coordinate systems. The observed skeleton is defined in image coordinates. The recognition skeleton is represented as normalized 3D landmarks. The canonical template is defined in a 3D template coordinate system. The retargeted hand model is manipulated in Blender world coordinates, and the final views are generated in virtual camera coordinates.

3.4.1 Observation and template coordinate systems

The observed skeleton used for fitting is represented in the image coordinate system. Each observed joint is a 2D vector

$$\mathbf{u}_i = \begin{bmatrix} u_i \\ v_i \end{bmatrix}.$$

The coordinate u_i corresponds to the horizontal image direction, while v_i corresponds to the vertical image direction. In many image processing systems, the vertical coordinate increases downward from the top of the image. The proposed fitting formulation is independent of the absolute unit of the coordinates because scale and translation are estimated during alignment.

The recognition skeleton is represented as a normalized 3D landmark set. This representation preserves relative depth and is used as the main input to the skeleton-based recognition model.

The canonical template is defined in a 3D coordinate system exported from Blender.

A template joint is represented as

$$\mathbf{X}_i = \begin{bmatrix} X_i \\ Y_i \\ Z_i \end{bmatrix} \in \mathbb{R}^3.$$

Since the original template may have arbitrary global translation in the Blender scene, the template is centered at the wrist before fitting:

$$\bar{\mathbf{X}}_i = \mathbf{X}_i - \mathbf{X}_0.$$

After this normalization, the wrist becomes the origin:

$$\bar{\mathbf{X}}_0 = \mathbf{0}.$$

The wrist-centered template $\bar{\mathcal{X}}$ is used as the input to the global alignment stage.

3.4.2 Blender world and rendering camera coordinate systems

The Blender world coordinate system is used during rig retargeting and rendering. When the fitted skeleton is loaded into Blender, it is placed relative to the actual hand armature. Bone directions and pose-bone transformations are evaluated in this coordinate system. Therefore, the target skeleton directions and current bone directions must be expressed consistently before bone rotation is computed.

The rendering camera coordinate system is used only after the hand mesh has been posed. Virtual cameras are placed around the hand to generate multi-view images and camera-space skeleton observations. This camera system is different from the weak-perspective projection model used during fitting. The fitting projection estimates an approximate relationship between the 3D template and the observed 2D skeleton, while the rendering cameras are used to synthesize new visual and geometric observations from the final 3D pose. Therefore, the rendering cameras are not required to reproduce the original input camera exactly. Their purpose is to generate controlled viewpoint diversity for synthetic multi-view supervision and visualization. Figure 3.4 summarizes these coordinate systems and their roles in the pipeline.

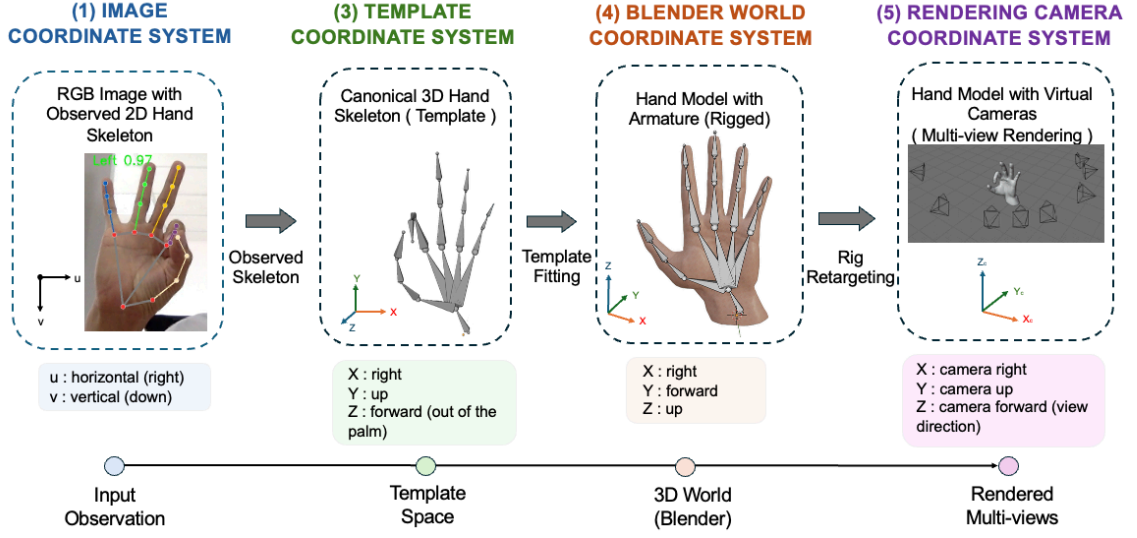


Figure 3.4: Coordinate systems used in the proposed pipeline: image coordinates, template coordinates, Blender world coordinates, and rendering camera coordinates.

3.5 Canonical 3D hand template

3.5.1 Motivation for using a canonical template

The proposed method uses a canonical 3D hand template as the geometric prior for reconstructing a plausible hand pose from a static 2D skeleton observation. This design is necessary because a 2D skeleton alone does not provide enough information to uniquely determine a full 3D hand configuration. Each observed landmark specifies only its location on the image plane, while its depth relative to the camera remains unknown. Therefore, directly recovering a complete 3D hand pose from a single 2D skeleton is an under-constrained problem.

This ambiguity can be explained from the standard perspective projection model. A 3D point $\mathbf{X} = [X, Y, Z]^T$ is projected to image coordinates according to $u = f_x X/Z + c_x$ and $v = f_y Y/Z + c_y$, where f_x and f_y are focal lengths and (c_x, c_y) is the camera principal point. These equations show that the projected coordinates depend on the ratios X/Z and Y/Z , rather than on X , Y , and Z independently. For example, the points $[1, 1, 2]^T$ and $[2, 2, 4]^T$ have different spatial positions, but they produce the same normalized projection because $1/2 = 2/4$. This simple example illustrates why depth cannot be uniquely recovered from a single image observation.

The ambiguity becomes more significant when the entire hand skeleton is considered. A 21-joint 3D skeleton contains $21 \times 3 = 63$ coordinate values, while the corresponding

2D skeleton provides only $21 \times 2 = 42$ coordinate values. Even when the 2D landmarks are accurately detected, the missing depth coordinates allow many possible 3D skeletons to produce similar 2D projections. Without additional geometric constraints, the reconstruction may result in physically implausible finger positions, unrealistic bone lengths, or inconsistent hand topology.

To reduce this ambiguity, the proposed method reformulates the reconstruction task as a template fitting problem. Instead of estimating all 3D joint coordinates independently, the system starts from a canonical 3D hand template that already contains valid hand proportions, a fixed skeleton topology, and compatibility with a rigged mesh model. The observed 2D skeleton is then used to guide the alignment and local refinement of this template. In this formulation, the template acts as a geometric prior that restricts the solution space and helps preserve anatomical consistency [16].

The canonical template is especially important because the pipeline is not limited to estimating joint coordinates. It also needs to produce geometrically plausible hand poses that can be visualized, retargeted to a rigged mesh, and observed from multiple virtual viewpoints. Once the template skeleton is fitted to the observed pose, the corresponding hand mesh can be posed and rendered under different camera and appearance conditions.

3.5.2 Template skeleton representation

The canonical hand template is constructed from a rigged 3D hand model in Blender. The model contains a hand mesh, an armature, and a set of bones that deform the mesh. From this rigged model, a 21-joint template skeleton is extracted so that the template has the same topology as the observed hand skeleton. This one-to-one correspondence is essential because the fitting objective compares each projected template joint with the corresponding observed 2D landmark. Figure 3.6 shows the rigged hand model and the extracted 21-joint template skeleton.

Let the template skeleton be denoted as

$$\mathcal{X} = \{\mathbf{X}_0, \mathbf{X}_1, \dots, \mathbf{X}_{20}\}, \quad \mathbf{X}_i = \begin{bmatrix} X_i \\ Y_i \\ Z_i \end{bmatrix} \in \mathbb{R}^3.$$

The index i follows the same 21-joint convention as the observed skeleton. Therefore, $\mathbf{X}_0$ corresponds to the wrist, $\mathbf{X}_1$ to $\mathbf{X}_4$ correspond to the thumb chain, $\mathbf{X}_5$ to $\mathbf{X}_8$ correspond to the index finger chain, and so on.

The extracted template skeleton must preserve the same kinematic structure as the

observed skeleton. If the template joint indexing is inconsistent with the observed skeleton indexing, the fitting loss would compare unrelated joints and the fitted pose would become invalid. For example, matching an index fingertip template joint to a middle fingertip observation would produce a numerical comparison without anatomical meaning. For this reason, the template skeleton is constructed with a deterministic joint mapping.

The original hand model may be located at an arbitrary position in the Blender scene. However, the absolute scene location of the template is not relevant to the fitting problem. The fitting stage is concerned with the relative hand structure and pose configuration. Therefore, the template is centered at the wrist joint before fitting:

$$\bar{\mathbf{X}}_i = \mathbf{X}_i - \mathbf{X}_0, \quad i = 0, \dots, 20.$$

This operation translates the entire skeleton so that the wrist becomes the origin. Since $\bar{\mathbf{X}}_0 = \mathbf{0}$, all other joints are represented relative to the wrist. The wrist-centered template is denoted as

$$\bar{\mathcal{X}} = \{\bar{\mathbf{X}}_0, \bar{\mathbf{X}}_1, \dots, \bar{\mathbf{X}}_{20}\}.$$

This representation removes arbitrary global translation inherited from the Blender scene and provides a consistent reference frame for template fitting. The wrist-centering step is illustrated in Figure 3.5.

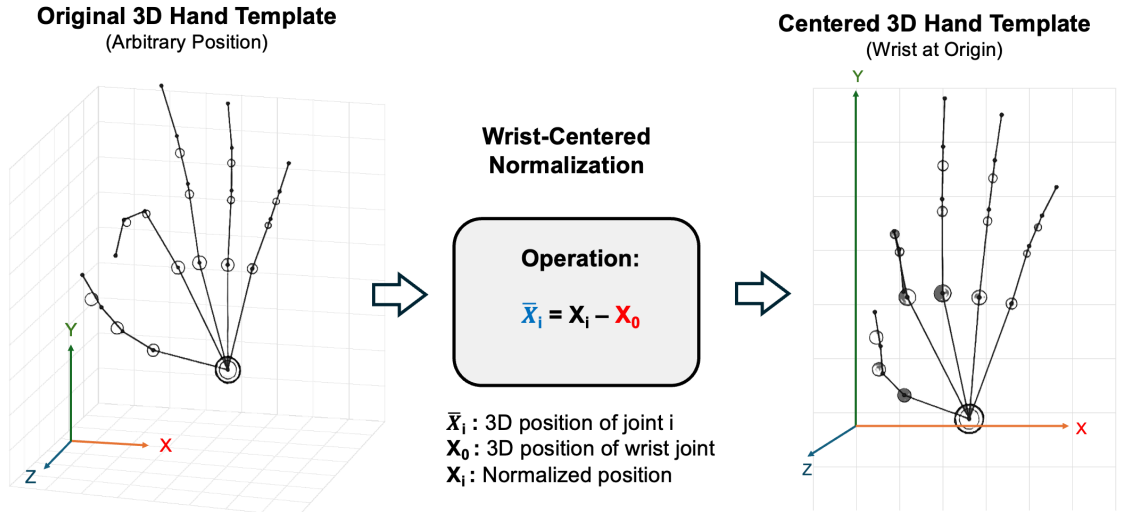


Figure 3.5: Wrist-centered template normalization. The original template skeleton is translated so that the wrist joint becomes the origin of the template coordinate system.

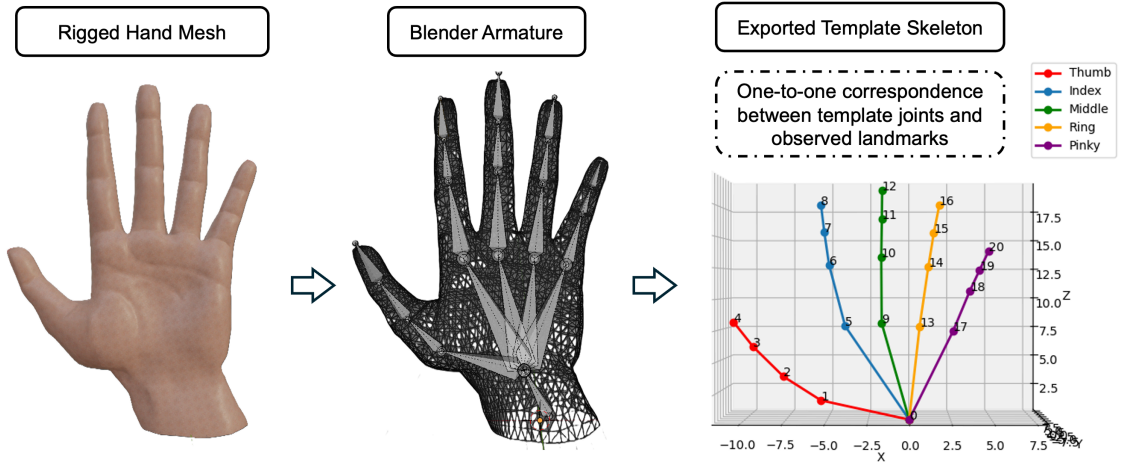


Figure 3.6: Canonical 3D hand template and its extracted 21-joint skeleton.

3.5.3 Template skeleton construction

The template skeleton is constructed by extracting selected anatomical joint locations from the Blender hand rig. Since the hand mesh is controlled by an armature, each skeleton landmark can be associated with a consistent location on the corresponding bone chain. The wrist landmark is obtained from the wrist-related bone endpoint, finger base landmarks are associated with metacarpal or proximal bone positions, and distal landmarks are associated with downstream phalangeal bones.

This construction step has two purposes. First, it ensures that the sparse 21-joint template skeleton has the same topology as the observed 2D skeleton. Second, it ensures that the fitted skeleton remains compatible with the rig used later during retargeting. If the extracted skeleton were not consistent with the rig structure, the fitted joint positions would be difficult to transfer back to the mesh.

The template construction is performed once before processing the dataset. The resulting template skeleton is reused for all input samples. During fitting, the template is not treated as a free set of points. Instead, it is treated as a structured hand model whose global pose and local articulation can be adjusted while preserving the original kinematic hierarchy.

3.6 Weak-perspective projection model

3.6.1 From perspective projection to weak-perspective projection

To compare the 3D template skeleton with the observed 2D skeleton, the 3D template must be projected onto the image plane. A physically accurate camera projection is described by the pinhole camera model. For a 3D point $\mathbf{X} = [X, Y, Z]^T$ expressed in camera coordinates, its image coordinates are given by

$$u = f_x \frac{X}{Z} + c_x, \quad v = f_y \frac{Y}{Z} + c_y.$$

In homogeneous form, this relationship can be written as

$$\lambda \begin{bmatrix} u \\ v \\ 1 \end{bmatrix} = \begin{bmatrix} f_x & 0 & c_x \\ 0 & f_y & c_y \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} X \\ Y \\ Z \end{bmatrix}.$$

Although this model is geometrically accurate, it requires camera intrinsic parameters and reliable depth information. These quantities are not assumed to be available in the proposed pipeline. Furthermore, the fitting stage only needs an approximate relationship between the 3D template and the 2D skeleton, not a full physical reconstruction of the original camera system.

For this reason, the proposed method adopts a weak-perspective projection model. The weak-perspective approximation assumes that the depth variation inside the hand is small compared with the average distance between the hand and the camera. Let Z_0 denote the average hand depth. If $Z_i \approx Z_0$ for all hand joints, then the perspective equations can be approximated as

$$u_i \approx \frac{f_x}{Z_0} X_i + c_x, \quad v_i \approx \frac{f_y}{Z_0} Y_i + c_y.$$

When the image axes are assumed to have approximately uniform scale, the two scale factors f_x/Z_0 and f_y/Z_0 can be represented by a single scalar s . The projection then becomes

$$u_i = sX_i + t_x, \quad v_i = sY_i + t_y. \quad (3.7)$$

Thus, the projection can be modeled as an orthographic projection followed by a two-dimensional similarity transformation.

This approximation is suitable for the proposed pipeline because the goal is not to recover the exact physical camera configuration of the input image. Instead, the goal is to

estimate a stable projection relationship that allows the 3D template skeleton to be aligned with the observed 2D skeleton. The weak-perspective model provides this relationship using only a rotation, a scale factor, and a translation vector.

3.6.2 Projection of the template skeleton

After the canonical template is centered at the wrist, each 3D joint is transformed by a candidate global rotation before being compared with the observed 2D skeleton. For the i -th template joint, the rotated coordinate is $\mathbf{Y}_i = \mathbf{R}\bar{\mathbf{X}}_i$, where $\mathbf{R} \in SO(3)$ represents a candidate orientation of the hand.

Because the observed skeleton is two-dimensional, the rotated 3D point is projected onto the image plane. In the weak-perspective setting, the orthographic projection operator $\Pi(\cdot)$ keeps only the first two coordinates of a 3D point:

$$\Pi \left(\begin{bmatrix} x \\ y \\ z \end{bmatrix} \right) = \begin{bmatrix} x \\ y \end{bmatrix}.$$

The projected template point before image-plane scale and translation is therefore $\mathbf{p}_i = \Pi(\mathbf{R}\bar{\mathbf{X}}_i)$. The projected point $\mathbf{p}_i$ is still expressed in the coordinate scale of the rotated 3D template. To compare it with the observed landmark $\mathbf{u}_i$, a scale factor and translation vector are applied. The final projected landmark is

$$\hat{\mathbf{u}}_i = s\mathbf{p}_i + \mathbf{t} = s\Pi(\mathbf{R}\bar{\mathbf{X}}_i) + \mathbf{t},$$

where $s > 0$ is a global scale factor and $\mathbf{t} \in \mathbb{R}^2$ is a two-dimensional translation vector. The three global alignment parameters are therefore $\mathbf{R}$, s , and $\mathbf{t}$.

This formulation separates the global alignment problem into interpretable components. The rotation controls the global hand orientation, the orthographic projection maps the rotated 3D skeleton to the image plane, the scale adapts the template size to the observed skeleton, and the translation aligns the projected skeleton with the observed image-plane location. Figure 3.7 illustrates how the 3D template is projected and aligned with the observed 2D skeleton.

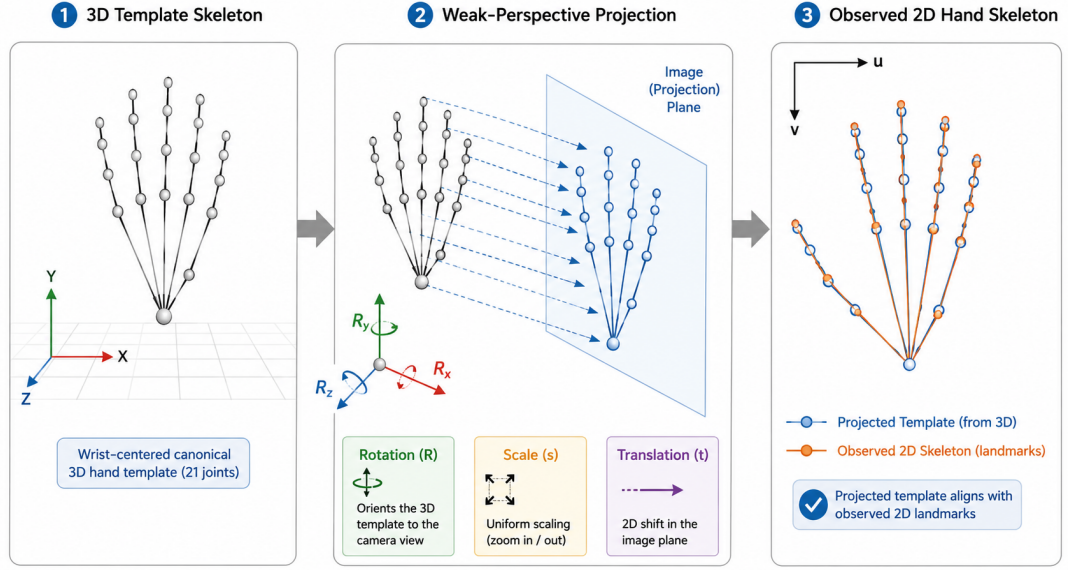


Figure 3.7: Weak-perspective projection used to align the canonical 3D template skeleton with the observed 2D skeleton.

3.7 Global template alignment

After defining the weak-perspective projection model, the next step is global template alignment. This stage estimates the overall hand orientation, scale, and image-plane location before local finger refinement.

The purpose of this stage is to establish a coarse geometric correspondence between the template skeleton and the target skeleton. At this point, the template still retains its original neutral finger configuration. Therefore, the objective of global alignment is not to perfectly match every joint. Instead, it aims to place the template in a viewpoint and spatial configuration that approximately corresponds to the observed hand pose.

Global alignment is necessary because the canonical template is defined in its own coordinate system, while the observed skeleton may correspond to a hand captured from an arbitrary camera viewpoint. Even if the template and the target represent the same gesture, their coordinates cannot be directly compared. The template must first be rotated, projected to the image plane, scaled to the correct size, and translated to the correct image location.

For a wrist-centered template joint $\bar{\mathbf{X}}_i$, the projected landmark after global transformation is defined as

$$\hat{\mathbf{u}}_i = s\Pi(\mathbf{R}\bar{\mathbf{X}}_i) + \mathbf{t},$$

where $\mathbf{R}$ denotes the global rotation matrix, s denotes a positive scale factor, and $\mathbf{t}$ denotes a two-dimensional translation vector. The unknown parameters of the global alignment stage are therefore $\mathbf{R}$, s , and $\mathbf{t}$.

The proposed method estimates these parameters in two steps. First, a discrete search is performed over candidate hand orientations. Second, for each candidate orientation, the optimal scale and translation are estimated using a closed-form least-squares solution.

3.7.1 Rotation matrix parameterization

The global orientation of the hand template is represented by a rotation matrix $\mathbf{R} \in SO(3)$, where $SO(3)$ denotes the special orthogonal group. A valid rotation matrix must satisfy

$$\mathbf{R}^T \mathbf{R} = \mathbf{I}, \quad \det(\mathbf{R}) = 1.$$

The first condition guarantees orthogonality, while the second condition prevents reflections and preserves orientation. In the proposed implementation, the rotation matrix is parameterized using Euler angles because this representation is intuitive and allows a straightforward viewpoint search procedure.

Let θ_x , θ_y , θ_z denote rotations around the x -axis, y -axis, and z -axis, respectively. The corresponding elementary rotation matrices are

$$\mathbf{R}_x(\theta_x) = \begin{bmatrix} 1 & 0 & 0 \\ 0 & \cos \theta_x & -\sin \theta_x \\ 0 & \sin \theta_x & \cos \theta_x \end{bmatrix}, \quad \mathbf{R}_y(\theta_y) = \begin{bmatrix} \cos \theta_y & 0 & \sin \theta_y \\ 0 & 1 & 0 \\ -\sin \theta_y & 0 & \cos \theta_y \end{bmatrix},$$

$$\mathbf{R}_z(\theta_z) = \begin{bmatrix} \cos \theta_z & -\sin \theta_z & 0 \\ \sin \theta_z & \cos \theta_z & 0 \\ 0 & 0 & 1 \end{bmatrix}.$$

The final rotation matrix is obtained by composing the three rotations:

$$\mathbf{R} = \mathbf{R}_z(\theta_z) \mathbf{R}_y(\theta_y) \mathbf{R}_x(\theta_x). \quad (3.8)$$

This ordering means that the template is first rotated around the x -axis, then around the y -axis, and finally around the z -axis.

Rather than optimizing these angles continuously, the proposed method evaluates a discrete set of candidate viewpoints. This strategy is computationally simple and avoids convergence problems that may occur in nonlinear optimization.

3.7.2 Palm anchor-based alignment

For a fixed candidate rotation matrix $\mathbf{R}$, each template joint is projected to the image plane:

$$\mathbf{p}_i = \Pi(\mathbf{R}\bar{\mathbf{X}}_i).$$

At this stage, the projected template and the observed skeleton may still differ in scale and position. To estimate these quantities, the proposed method uses a subset of stable anchor joints:

$$\mathcal{A} = \mathcal{I}_{palm} = \{0, 5, 9, 13, 17\}.$$

This subset consists of the wrist joint and the base joints of the four non-thumb fingers. Palm anchors are selected because they are generally less affected by local finger articulation than fingertip joints. Consequently, they provide a more reliable reference for estimating global hand orientation and location.

The alignment objective is formulated as

$$E(s, \mathbf{t}) = \sum_{i \in \mathcal{A}} \|s\mathbf{p}_i + \mathbf{t} - \mathbf{u}_i\|_2^2. \quad (3.9)$$

The goal is to find the scale factor s and translation vector $\mathbf{t}$ that minimize this objective. The centroid of the projected anchor set is

$$\bar{\mathbf{p}} = \frac{1}{|\mathcal{A}|} \sum_{i \in \mathcal{A}} \mathbf{p}_i,$$

while the centroid of the observed anchor set is

$$\bar{\mathbf{u}} = \frac{1}{|\mathcal{A}|} \sum_{i \in \mathcal{A}} \mathbf{u}_i.$$

For a given scale s , the optimal translation must align the two centroids:

$$\mathbf{t}^* = \bar{\mathbf{u}} - s\bar{\mathbf{p}}. \quad (3.10)$$

This result can be obtained by differentiating the least-squares objective with respect to $\mathbf{t}$ and setting the derivative to zero. Substituting this translation into the original objective eliminates the translation variable and produces a scale-only optimization problem.

Define the centered coordinates:

$$\tilde{\mathbf{p}}_i = \mathbf{p}_i - \bar{\mathbf{p}}, \quad \tilde{\mathbf{u}}_i = \mathbf{u}_i - \bar{\mathbf{u}}.$$

The objective becomes

$$E(s) = \sum_{i \in \mathcal{A}} \|s\tilde{\mathbf{p}}_i - \tilde{\mathbf{u}}_i\|_2^2.$$

Expanding the squared norm yields

$$E(s) = \sum_{i \in \mathcal{A}} (s^2 \|\tilde{\mathbf{p}}_i\|_2^2 - 2s\tilde{\mathbf{p}}_i^T \tilde{\mathbf{u}}_i + \|\tilde{\mathbf{u}}_i\|_2^2).$$

Differentiating with respect to s gives

$$\frac{dE}{ds} = 2s \sum_{i \in \mathcal{A}} \|\tilde{\mathbf{p}}_i\|_2^2 - 2 \sum_{i \in \mathcal{A}} \tilde{\mathbf{p}}_i^T \tilde{\mathbf{u}}_i.$$

Setting the derivative to zero produces the optimal scale estimate

$$s^* = \frac{\sum_{i \in \mathcal{A}} \tilde{\mathbf{p}}_i^T \tilde{\mathbf{u}}_i}{\sum_{i \in \mathcal{A}} \|\tilde{\mathbf{p}}_i\|_2^2}.$$

The corresponding translation is then

$$\mathbf{t}^* = \bar{\mathbf{u}} - s^*\bar{\mathbf{p}}.$$

Only positive scale values are accepted: $s^* > 0$. A negative scale would correspond to a mirrored solution and would violate the anatomical orientation of the template hand.

3.7.3 Global viewpoint selection

The previous subsection provides the optimal scale and translation for a fixed rotation candidate. However, the global hand orientation is still unknown because multiple rotation candidates may produce different projected skeletons. Therefore, a viewpoint selection criterion is required to determine the best global alignment between the canonical template and the observed skeleton.

For each candidate rotation $\mathbf{R}$, the template skeleton is projected and aligned using the corresponding optimal scale $s^*(\mathbf{R})$ and translation $\mathbf{t}^*(\mathbf{R})$. The aligned projection of the i -th joint is computed as

$$\hat{\mathbf{u}}_i = s^*(\mathbf{R})\Pi(\mathbf{R}\bar{\mathbf{X}}_i) + \mathbf{t}^*(\mathbf{R}). \quad (3.11)$$

The projected skeleton is compared with the observed skeleton using three complementary error measures. The all-joint error evaluates the average discrepancy over the complete hand skeleton:

$$L_{all} = \frac{1}{|\mathcal{I}_{all}|} \sum_{i \in \mathcal{I}_{all}} \|\hat{\mathbf{u}}_i - \mathbf{u}_i\|_2. \quad (3.12)$$

The fingertip error evaluates only the fingertip joints:

$$L_{tip} = \frac{1}{|\mathcal{I}_{tip}|} \sum_{i \in \mathcal{I}_{tip}} \|\hat{\mathbf{u}}_i - \mathbf{u}_i\|_2. \quad (3.13)$$

The palm error evaluates the palm anchor joints:

$$L_{palm} = \frac{1}{|\mathcal{I}_{palm}|} \sum_{i \in \mathcal{I}_{palm}} \|\hat{\mathbf{u}}_i - \mathbf{u}_i\|_2. \quad (3.14)$$

These three terms capture different aspects of alignment quality. The all-joint term measures overall skeleton consistency, the fingertip term emphasizes gesture-defining landmarks, and the palm term stabilizes the global hand structure.

For later stages, the three error terms are combined into a general fitting loss:

$$L_{fit} = \lambda_{all}L_{all} + \lambda_{tip}L_{tip} + \lambda_{palm}L_{palm}. \quad (3.15)$$

This loss measures the overall quality of a projected skeleton with respect to the observed 2D skeleton. It is later reused during hierarchical refinement to evaluate candidate local transformations.

During global viewpoint selection, a separate viewpoint score is used:

$$L_{view} = \alpha_{all}L_{all} + \alpha_{tip}L_{tip} + \alpha_{palm}L_{palm}. \quad (3.16)$$

Although L_{view} and L_{fit} are based on the same error components, they serve different roles. The viewpoint score is used only to select the best global rotation, while the fitting loss is used as a general skeleton matching objective during both global evaluation and local refinement. In practice, the viewpoint score can assign a larger weight to palm anchors to obtain a stable global pose, whereas the fitting loss can assign stronger importance to fingertip accuracy for gesture-specific refinement.

The optimal global alignment is selected by minimizing the viewpoint score over the candidate rotation set:

$$\mathbf{R}^* = \arg \min_{\mathbf{R} \in \mathcal{R}_{cand}} L_{view}(\mathbf{R}, s^*(\mathbf{R}), \mathbf{t}^*(\mathbf{R})). \quad (3.17)$$

After the best rotation $\mathbf{R}^*$ is selected, the corresponding scale and translation are retained as $s^* = s^*(\mathbf{R}^*)$ and $\mathbf{t}^* = \mathbf{t}^*(\mathbf{R}^*)$.

The resulting parameters $(\mathbf{R}^*, s^*, \mathbf{t}^*)$ provide the initial global estimate of the hand pose. Although this estimate aligns the template skeleton with the observed hand at the global level, local finger articulation may still deviate from the target pose. Consequently, the next section introduces the hierarchical skeleton refinement procedure, which improves local finger alignment while preserving the anatomical structure of the hand. Algorithm 1 summarizes the global alignment procedure.

Algorithm 1 Global Template Alignment

Require: Observed skeleton $\mathcal{U}$; wrist-centered template skeleton $\bar{\mathcal{X}}$; candidate rotation set $\mathcal{R}_{cand}$; palm anchor set $\mathcal{I}_{palm}$

Ensure: Optimal global parameters $(\mathbf{R}^*, s^*, \mathbf{t}^*)$

```
1: Initialize best score  $L_{view}^* \leftarrow +\infty$ 
2: for each candidate rotation  $\mathbf{R} \in \mathcal{R}_{cand}$  do
3:   Project template joints:  $\mathbf{p}_i \leftarrow \Pi(\mathbf{R}\bar{\mathbf{X}}_i)$ 
4:   Estimate optimal scale  $s^*(\mathbf{R})$  using palm anchors
5:   Estimate optimal translation  $\mathbf{t}^*(\mathbf{R})$ 
6:   Compute aligned projection  $\hat{\mathbf{u}}_i$ 
7:   Compute  $L_{all}$ ,  $L_{tip}$ , and  $L_{palm}$ 
8:   Compute viewpoint score  $L_{view}$ 
9:   if  $L_{view} < L_{view}^*$  then
10:     $L_{view}^* \leftarrow L_{view}$ 
11:     $\mathbf{R}^* \leftarrow \mathbf{R}$ 
12:     $s^* \leftarrow s^*(\mathbf{R})$ 
13:     $\mathbf{t}^* \leftarrow \mathbf{t}^*(\mathbf{R})$ 
14:   end if
15: end for
16: return  $(\mathbf{R}^*, s^*, \mathbf{t}^*)$ 
```

3.8 Hierarchical skeleton refinement

Global alignment provides a coarse match between the 3D template and the observed 2D skeleton. However, one global transformation cannot capture local finger articulation. Since static gestures often depend on finger bending, fingertip placement, and finger spacing, hierarchical refinement is applied after global alignment.

The refinement stage operates directly in the 3D template space. Instead of moving each joint independently, which may break the anatomical continuity of the hand, the method applies local rotations around selected pivot joints. Each local rotation affects only the downstream joints of the corresponding finger chain. This design preserves the parent-child structure of the skeleton and keeps the refined pose compatible with the Blender rig used in the later retargeting stage.

Let the globally aligned template skeleton be initialized as $\mathcal{X}^{(0)} = \bar{\mathcal{X}}$. During refinement, the skeleton is updated iteratively. At the k -th update step, the current skeleton is denoted by $\mathcal{X}^{(k)}$, and the next skeleton is obtained by applying a selected local transformation to a specific refinement group.

3.8.1 Motivation for local refinement

The purpose of global alignment is to estimate the coarse pose relationship between the 3D template and the observed 2D skeleton. Since the same rotation, scale, and translation are applied to the entire template, this stage can align the palm region and approximate the hand viewpoint, but it cannot capture the full articulation of each finger.

This limitation is especially visible in gestures where the semantic meaning is determined by fingertip arrangement. In an OK gesture, for example, the thumb and index finger form a circular or nearly circular contact pattern, while the remaining fingers are usually extended or slightly bent. A neutral template may have the correct global orientation after alignment, but its thumb and index fingers may still fail to match the observed fingertip positions. Local refinement is therefore necessary to adapt the template to the specific gesture configuration.

The refinement stage also compensates for differences between the canonical template proportions and the observed skeleton. Even when both skeletons follow the same 21-joint topology, the exact finger lengths, palm width, and finger spacing may differ. The objective is not to reconstruct the physical hand of the original subject exactly, but to obtain a geometrically plausible 3D pose that is consistent with the observed 2D gesture and can be rendered as a realistic hand image.

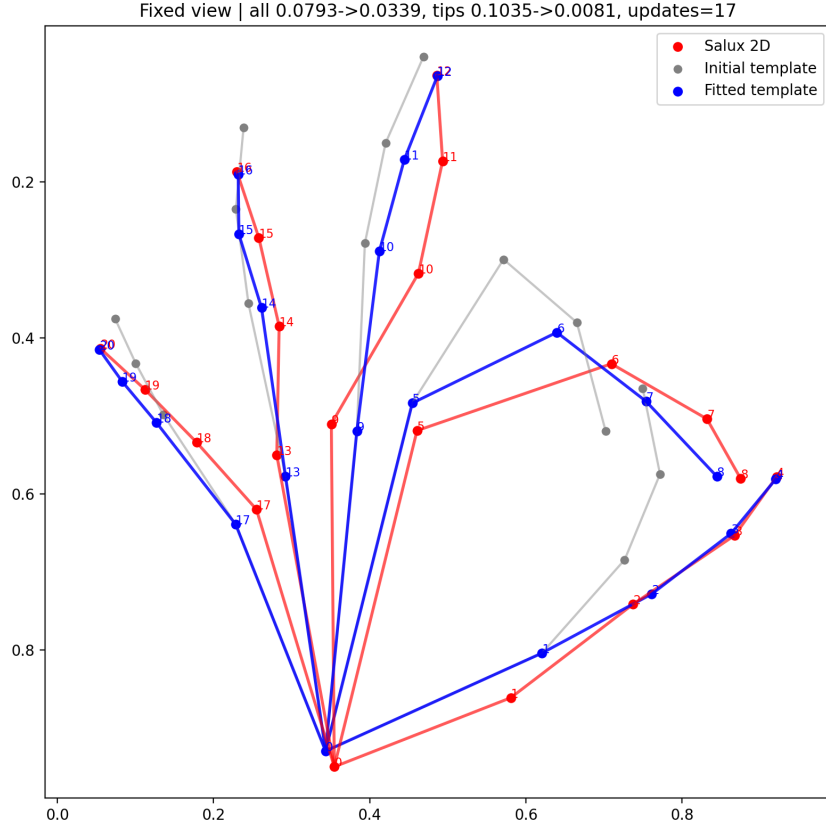


Figure 3.8: Example of template fitting before and after hierarchical refinement. The observed 2D skeleton, initial template projection, and refined template projection are visualized for comparison.

3.8.2 Pivot-based local transformation

Each local refinement operation is defined by a pivot joint, a set of affected downstream joints, and a candidate local rotation. The pivot joint acts as the local rotation center. The affected joints are rotated around this pivot, while the pivot itself and unrelated joints remain unchanged.

Consider a refinement group q . Let j_q denote the pivot joint of the group, and let $\mathcal{D}_q$ denote the set of affected downstream joints. For an affected joint $j \in \mathcal{D}_q$, the vector from the pivot to the joint is

$$\mathbf{v}_j^{(k)} = \mathbf{X}_j^{(k)} - \mathbf{X}_{j_q}^{(k)}.$$

Given a candidate local rotation matrix $\mathbf{R}_{local}(a, \theta)$, where $a \in \{x, y, z\}$ is the rotation axis and θ is the rotation angle, the candidate joint position is computed as

$$\mathbf{X}_j^{cand} = \mathbf{X}_{j_q}^{(k)} + \mathbf{R}_{local}(a, \theta) \left(\mathbf{X}_j^{(k)} - \mathbf{X}_{j_q}^{(k)} \right), \quad j \in \mathcal{D}_q. \quad (3.18)$$

For joints that are not affected by the current group, the candidate skeleton keeps their current positions:

$$\mathbf{X}_j^{cand} = \mathbf{X}_j^{(k)}, \quad j \notin \mathcal{D}_q.$$

This transformation can be written compactly as

$$\mathbf{X}_j^{cand} = \begin{cases} \mathbf{X}_{j_q}^{(k)} + \mathbf{R}_{local}(a, \theta) \left(\mathbf{X}_j^{(k)} - \mathbf{X}_{j_q}^{(k)} \right), & j \in \mathcal{D}_q, \\ \mathbf{X}_j^{(k)}, & j \notin \mathcal{D}_q. \end{cases} \quad (3.19)$$

The formulation keeps the pivot fixed and transforms only the downstream part of the finger. This is consistent with the kinematic interpretation of the hand skeleton, where rotating a parent joint influences its child joints but does not move unrelated fingers. Figure 3.9 illustrates this pivot-based local transformation.

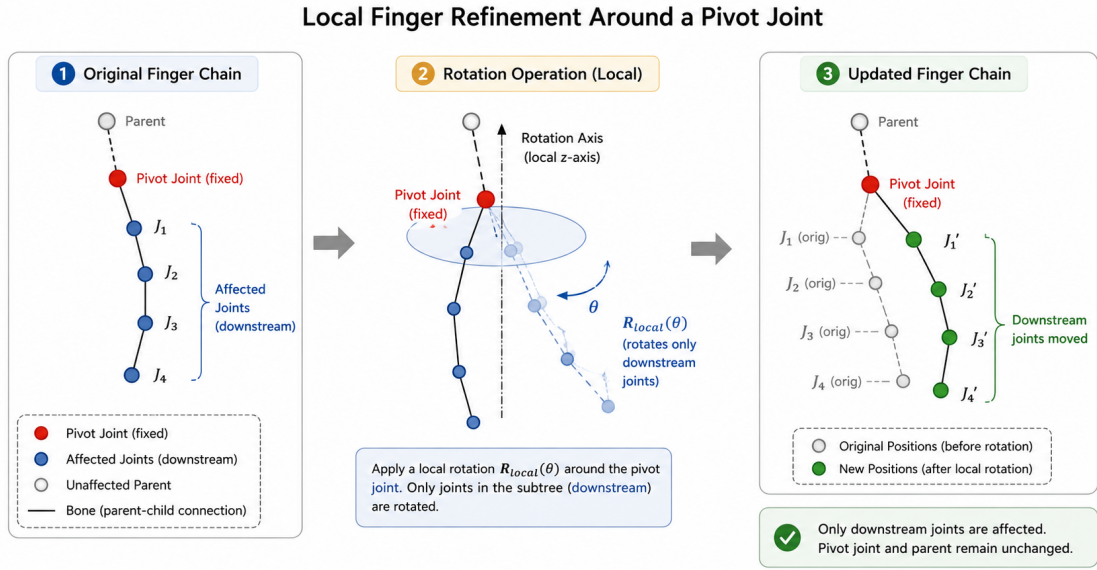


Figure 3.9: Local rotation around a pivot joint. Downstream joints are transformed around the pivot, while the pivot and unrelated joints remain fixed.

3.8.3 Finger refinement groups

The refinement process is organized into finger-specific groups. Each group contains a pivot joint and its corresponding downstream joints. For non-thumb fingers, the refinement follows the anatomical order from the metacarpophalangeal region toward the fingertip. The thumb is refined using the same parent-to-child principle according to its own chain structure.

Figure 3.10 illustrates the refinement group convention used in this thesis. Each finger group is defined by a pivot joint and the affected joints that receive the propagated local transformation. The arrows indicate the direction of influence propagation along the finger chain.

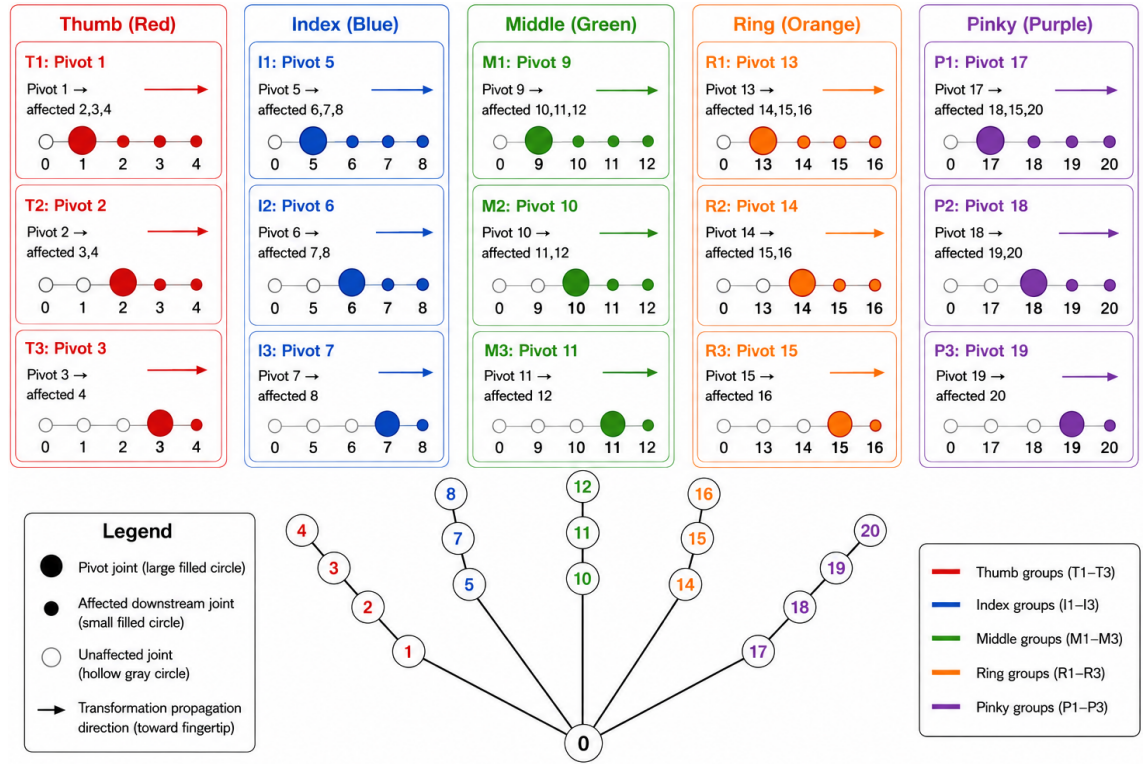


Figure 3.10: Hierarchical finger refinement groups. Each refinement group consists of a pivot joint and its affected downstream joints. Transformations propagate from parent joints toward distal joints following the anatomical finger hierarchy.

The use of refinement groups prevents independent and unconstrained joint displacement. For example, when the index MCP joint is selected as a pivot, the index PIP, DIP, and fingertip joints are affected together. When the index PIP joint is selected as a pivot, only the DIP and fingertip are affected. This grouping strategy ensures that each finger remains a connected kinematic chain [12] throughout refinement.

3.8.4 Local refinement objective

For each candidate local rotation, the candidate skeleton is projected back to the image plane using the global projection parameters $(\mathbf{R}^*, s^*, \mathbf{t}^*)$ obtained from the global alignment stage:

$$\hat{\mathbf{u}}_i^{cand} = s^* \Pi(\mathbf{R}^* \mathbf{X}_i^{cand}) + \mathbf{t}^*.$$

The global parameters are kept fixed during local refinement. This means that refinement changes only the internal 3D articulation of the template skeleton, not the estimated global viewpoint or image-plane placement.

The candidate is evaluated using a combination of global skeleton consistency, local group accuracy, and regularization. Let $\mathcal{G}_q$ denote the target joints used to evaluate the current group. The local group error is defined as

$$L_{local}^q = \frac{1}{|\mathcal{G}_q|} \sum_{i \in \mathcal{G}_q} \|\hat{\mathbf{u}}_i^{cand} - \mathbf{u}_i\|_2. \quad (3.20)$$

This term measures whether the current local transformation improves the affected part of the finger.

The global fitting loss for the candidate skeleton is computed using the fitting loss defined in Equation 3.15. For the candidate skeleton $\mathcal{X}^{cand}$, this loss is denoted as $L_{fit}(\mathcal{X}^{cand})$. It prevents the refinement from improving one local group while damaging the overall hand structure.

Because the input observation is two-dimensional, different 3D configurations may produce similar 2D projections. To reduce unnatural rotations, a regularization term is added:

$$L_{reg} = \gamma |\theta|, \quad (3.21)$$

where θ is the candidate rotation angle and γ is a small positive coefficient.

The final refinement score for group q and candidate rotation (a, θ) is

$$L_{refine}^q(a, \theta) = L_{fit}(\mathcal{X}^{cand}) + \beta L_{local}^q + L_{reg}. \quad (3.22)$$

The coefficient β controls the contribution of the local group error. A larger value of

β gives more priority to the currently refined finger group, while a smaller value makes the optimization more conservative.

For the current refinement group, the best candidate is selected by minimizing the refinement score:

$$(a^*, \theta^*) = \arg \min_{(a, \theta) \in \Omega} L_{refine}^q(a, \theta), \quad (3.23)$$

where Ω denotes the discrete set of candidate axes and angles. The current skeleton is then updated using this selected local transformation.

3.8.5 Parent-to-child refinement order

The order of refinement is important because the skeleton is hierarchical. A transformation applied to a parent joint changes the positions of all downstream joints, while a transformation applied to a distal joint affects only a smaller part of the finger. Therefore, each finger is refined from the proximal region toward the fingertip.

For non-thumb fingers, the refinement order follows

$$MCP \rightarrow PIP \rightarrow DIP.$$

When the MCP group is refined, the PIP, DIP, and fingertip joints are affected. When the PIP group is refined, only the DIP and fingertip are affected. When the DIP group is refined, only the fingertip is affected. This order is consistent with the kinematic dependency of finger joints.

If the order were reversed, later parent-joint updates could invalidate earlier distal-joint refinements. For instance, correcting a fingertip first and then rotating the MCP joint would move the entire downstream chain, changing the fingertip position again. The parent-to-child strategy avoids this instability and provides a more reliable refinement sequence.

3.8.6 Hierarchical refinement procedure

The previous subsections define the three essential components of the hierarchical refinement stage: the pivot-based local transformation, the finger refinement groups, and the local refinement objective. This subsection describes how these components are combined into a complete refinement procedure.

The refinement starts from the globally aligned template skeleton. The global projection parameters ($\mathbf{R}^*$, s^* , $\mathbf{t}^*$) are kept fixed throughout this stage. Therefore, the refinement process does not re-estimate the global viewpoint, scale, or image-plane translation. Instead, it only updates the internal articulation of the 3D template skeleton.

For each refinement group, the procedure identifies its pivot joint and affected downstream joints according to the group definition illustrated in Figure 3.10. A finite set of candidate rotations is then evaluated around the pivot. Each candidate produces a temporary skeleton configuration by modifying only the affected joints of the current group. The temporary skeleton is projected to the image plane using the fixed global projection parameters and evaluated using the refinement objective defined in Section 3.8.4.

The candidate with the lowest refinement score is selected as the local update for the current group. If no candidate provides an improvement over the current configuration, the skeleton is left unchanged for that group. After this decision is made, the algorithm proceeds to the next refinement group following the parent-to-child order described in Section 3.8.5.

A complete pass is obtained after all refinement groups have been processed once. Since a transformation applied to a proximal joint may change the optimal configuration of downstream joints, the procedure is repeated for a small fixed number of passes. In each pass, the same parent-to-child traversal order is used. This iterative strategy allows distal joints to adapt to earlier proximal updates while preventing excessive deformation of the template skeleton.

The output of the refinement stage is the fitted 3D skeleton

$$\mathcal{X}^{fit} = \{\mathbf{X}_0^{fit}, \mathbf{X}_1^{fit}, \dots, \mathbf{X}_{20}^{fit}\}.$$

This skeleton preserves the 21-joint topology of the canonical template while adapting local finger articulation to the observed 2D pose. Because the refinement is performed through hierarchical rotations rather than independent joint displacement, the fitted skeleton remains anatomically consistent and compatible with the Blender rig used in the subsequent retargeting stage. Algorithm 2 summarizes the hierarchical refinement procedure.

Algorithm 2 Hierarchical Skeleton Refinement

Require: Initial template skeleton $\bar{\mathcal{X}}$; observed skeleton $\mathcal{U}$; global parameters $(\mathbf{R}^*, s^*, \mathbf{t}^*)$; refinement groups $\mathcal{Q}$; candidate rotation set Ω ; number of passes P

Ensure: Refined skeleton $\mathcal{X}^{fit}$

```
1: Initialize  $\mathcal{X}^{(0)} \leftarrow \bar{\mathcal{X}}$ 
2: for  $p = 1$  to  $P$  do
3:   for each refinement group  $q \in \mathcal{Q}$  in parent-to-child order do
4:     Identify pivot joint  $j_q$  and downstream joint set  $\mathcal{D}_q$ 
5:     Initialize best score  $L_{refine}^* \leftarrow L_{fit}(\mathcal{X}^{(k)})$ 
6:     Initialize best skeleton  $\mathcal{X}^{best} \leftarrow \mathcal{X}^{(k)}$ 
7:     for each candidate rotation  $(a, \theta) \in \Omega$  do
8:       Generate candidate skeleton  $\mathcal{X}^{cand}$  by rotating downstream joints around
        $j_q$ 
9:       Project  $\mathcal{X}^{cand}$  using  $(\mathbf{R}^*, s^*, \mathbf{t}^*)$ 
10:      Compute refinement score  $L_{refine}^q(a, \theta)$ 
11:      if  $L_{refine}^q(a, \theta) < L_{refine}^*$  then
12:         $L_{refine}^* \leftarrow L_{refine}^q(a, \theta)$ 
13:         $\mathcal{X}^{best} \leftarrow \mathcal{X}^{cand}$ 
14:      end if
15:    end for
16:    Update current skeleton  $\mathcal{X}^{(k+1)} \leftarrow \mathcal{X}^{best}$ 
17:  end for
18: end for
19:  $\mathcal{X}^{fit} \leftarrow \mathcal{X}^{(k)}$ 
20: return  $\mathcal{X}^{fit}$ 
```

3.9 Retargeting to the Blender hand rig

After hierarchical refinement, the output is a refined 3D skeleton $\mathcal{X}^{fit}$. This skeleton stores joint coordinates, but Blender hand models are controlled by bones. Therefore, the fitted skeleton must be transferred to a rigged 3D hand mesh. This step is called rig retargeting.

The purpose of retargeting is to convert the refined skeleton into bone rotations that can drive the hand mesh. Instead of directly assigning absolute joint coordinates to the mesh, the proposed method aligns the direction of each rig bone with the corresponding segment direction of the fitted skeleton. This design is compatible with standard armature-based deformation, where the mesh is controlled by bone transformations rather than by independent movement of surface vertices.

Let the refined skeleton be denoted by $\mathcal{X}^{fit} = \{\mathbf{X}_0^{fit}, \mathbf{X}_1^{fit}, \dots, \mathbf{X}_{20}^{fit}\}$. Each pair of connected joints defines a skeleton segment. For example, the segment from joint 5 to joint 6 corresponds to the proximal part of the index finger. In the Blender rig, this segment is

associated with a specific pose bone. Retargeting is performed by rotating the pose bone so that its current direction becomes aligned with the target direction computed from the fitted skeleton.

3.9.1 Skeleton segment direction

For a skeleton segment connecting a parent joint j_0 and a child joint j_1 , the target segment direction is obtained from the fitted skeleton. The unnormalized segment vector is $\mathbf{X}_{j_1}^{fit} - \mathbf{X}_{j_0}^{fit}$. Since retargeting only requires direction alignment, this vector is normalized to obtain a unit direction.

$$\mathbf{d}_{j_0,j_1}^{target} = \frac{\mathbf{X}_{j_1}^{fit} - \mathbf{X}_{j_0}^{fit}}{\|\mathbf{X}_{j_1}^{fit} - \mathbf{X}_{j_0}^{fit}\|_2}. \quad (3.24)$$

The direction $\mathbf{d}_{j_0,j_1}^{target}$ represents the desired orientation of the corresponding bone after retargeting. The normalization step is important because bone length in the Blender rig is determined by the rig itself, while the fitting result may have different scale or proportions. By aligning directions rather than absolute endpoints, the method preserves the original rig structure and avoids unrealistic stretching of the mesh.

3.9.2 Bone direction in the Blender rig

In Blender, each pose bone has a local coordinate frame. In the implemented hand rig, the local Y -axis of a pose bone is used as the bone direction. Therefore, for a pose bone b , its current direction in world coordinates can be represented as $\mathbf{d}_b^{bone}$. This direction is computed from the current pose-bone orientation and the armature world transformation.

The objective of retargeting is to rotate each pose bone so that its current direction $\mathbf{d}_b^{bone}$ becomes aligned with the target skeleton direction $\mathbf{d}_{j_0,j_1}^{target}$. For clarity, let $\mathbf{a}$ denote the current bone direction and $\mathbf{b}$ denote the target skeleton direction. Both vectors are normalized before computing the alignment rotation.

$$\mathbf{a} = \frac{\mathbf{d}_b^{bone}}{\|\mathbf{d}_b^{bone}\|_2}, \quad \mathbf{b} = \frac{\mathbf{d}_{j_0,j_1}^{target}}{\|\mathbf{d}_{j_0,j_1}^{target}\|_2}. \quad (3.25)$$

This direction-based formulation is more stable than attempting to directly match all bone endpoints. A rigged hand mesh has predefined bone lengths, skinning weights, and local coordinate frames. Forcing the mesh to match arbitrary 3D joint locations may distort

the rig, whereas aligning bone directions preserves the intended deformation behavior of the hand model.

3.9.3 Quaternion-based direction alignment

To rotate the current bone direction $\mathbf{a}$ onto the target direction $\mathbf{b}$, the method computes the shortest rotation between the two unit vectors. The rotation axis is determined by the cross product $\mathbf{a} \times \mathbf{b}$, and the rotation angle is determined by the dot product $\mathbf{a}^T \mathbf{b}$.

$$\mathbf{k} = \mathbf{a} \times \mathbf{b}, \quad \theta = \cos^{-1}(\mathbf{a}^T \mathbf{b}). \quad (3.26)$$

If $\mathbf{k}$ is non-zero, the unit rotation axis is obtained as $\hat{\mathbf{k}} = \mathbf{k} / \|\mathbf{k}\|_2$. The corresponding unit quaternion is then written as

$$\mathbf{q} = \begin{bmatrix} \cos(\theta/2) \\ \hat{k}_x \sin(\theta/2) \\ \hat{k}_y \sin(\theta/2) \\ \hat{k}_z \sin(\theta/2) \end{bmatrix}. \quad (3.27)$$

As illustrated in Figure 3.11, the alignment rotation is computed as the shortest rotation that transforms the current bone direction into the target skeleton direction. This formulation provides a stable and geometrically meaningful way to transfer pose information from the fitted skeleton to the Blender rig.

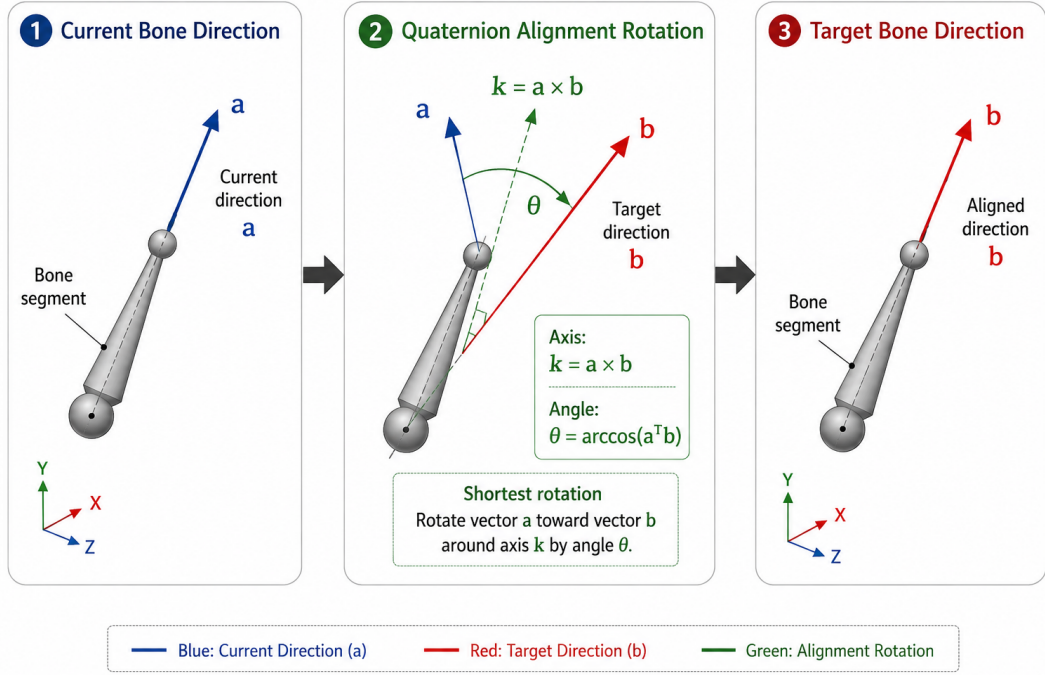


Figure 3.11: Quaternion-based bone direction alignment.

The quaternion q represents the rotation that aligns the current bone direction with the target skeleton segment direction. In practice, numerical edge cases must be handled carefully. If the two directions are already nearly identical, the rotation angle is close to zero and the bone can be left unchanged. If the two directions are nearly opposite, the cross product may become unstable, and an alternative orthogonal axis may be selected. These cases are implementation details, but they are important for stable batch processing.

3.9.4 Pose bone matrix update

After the alignment rotation has been computed, it must be applied to the corresponding pose bone. The rotation should be performed around the bone head rather than around the global origin. This is necessary because a finger bone should rotate around its anatomical joint. Rotating around the world origin would introduce incorrect translation and destroy the local kinematic behavior of the finger.

Let H_b denote the head position of bone b , and let M_b denote the current pose-bone transformation matrix. The rotation matrix corresponding to the quaternion q is denoted by $R(q)$. The updated pose-bone matrix is computed as

$$M'_b = T(H_b)R(q)T(-H_b)M_b. \quad (3.28)$$

Here, $T(\mathbf{H}_b)$ is a translation matrix that moves the coordinate frame to the bone head, and $T(-\mathbf{H}_b)$ moves it back after rotation. This transformation ensures that the bone rotates around its own head while preserving the hierarchical relationship between parent and child bones.

The retargeting process is applied in parent-to-child order. This order is necessary because child bones inherit transformations from their parent bones. If a distal bone were aligned before its parent, the later parent update would change the child direction and invalidate the previous alignment. Therefore, the same hierarchical principle used in skeleton refinement is also used during rig retargeting.

3.9.5 Joint-to-bone correspondence

The fitted skeleton and the Blender rig are connected through a deterministic joint-to-bone correspondence. Each skeleton segment is associated with one pose bone in the armature. This mapping allows the system to compute a target direction from the fitted skeleton and apply it to the correct rig bone.

Table 3.1: Correspondence between fitted skeleton segments and Blender hand rig bones.

Finger	Skeleton segment	Blender bone
Thumb	$0 \rightarrow 1$	thumb_trapez
Thumb	$1 \rightarrow 2$	thumb_meta
Thumb	$2 \rightarrow 3$	thumb_prox
Thumb	$3 \rightarrow 4$	thumb_dist
Index	$0 \rightarrow 5$	index_meta
Index	$5 \rightarrow 6$	index_prox
Index	$6 \rightarrow 7$	index_midd
Index	$7 \rightarrow 8$	index_dist
Middle	$0 \rightarrow 9$	midd_meta
Middle	$9 \rightarrow 10$	midd_prox
Middle	$10 \rightarrow 11$	midd_midd
Middle	$11 \rightarrow 12$	midd_dist
Ring	$0 \rightarrow 13$	ring_meta
Ring	$13 \rightarrow 14$	ring_prox
Ring	$14 \rightarrow 15$	ring_midd
Ring	$15 \rightarrow 16$	ring_dist
Pinky	$0 \rightarrow 17$	pinky_meta
Pinky	$17 \rightarrow 18$	pinky_prox
Pinky	$18 \rightarrow 19$	pinky_midd
Pinky	$19 \rightarrow 20$	pinky_dist

The mapping in Table 3.1 follows the same skeleton topology used throughout the

proposed method. This consistency is important because any mismatch between skeleton segments and rig bones would result in incorrect mesh deformation. For example, if an index finger segment were assigned to a middle finger bone, the rendered hand pose would no longer correspond to the fitted skeleton.

3.9.6 Retargeting procedure

The retargeting procedure begins by clearing the previous pose of the hand rig. The fitted skeleton $\mathcal{X}^{fit}$ is then loaded and placed relative to the wrist location of the Blender armature. For each skeleton segment, the target direction is computed using Equation 3.24. The current direction of the corresponding pose bone is then obtained from the rig. A quaternion alignment rotation is computed and applied to the pose-bone matrix using Equation 3.28.

The procedure is applied sequentially from the wrist and palm region toward the fingertips. After all mapped bones have been updated, the hand mesh follows the fitted skeleton pose through the armature deformation system. The resulting posed mesh is denoted as $\mathcal{M}^{pose}$.

$$\mathcal{M}^{pose} = \mathcal{R}_{rig}(\mathcal{M}, \mathcal{X}^{fit}). \quad (3.29)$$

In this notation, $\mathcal{M}$ is the original rigged hand mesh and $\mathcal{R}_{rig}(\cdot)$ denotes the rig retargeting operation. This notation summarizes the role of retargeting: it transforms the refined skeleton into a posed 3D hand mesh that can be rendered from different viewpoints.

The retargeting stage completes the transition from sparse geometric representation to dense mesh representation. Algorithm 3 summarizes the direction-based retargeting procedure. The fitted skeleton provides the pose, while the rigged mesh provides the visible hand surface. This posed mesh is the direct input to the synthetic rendering stage described in the next section.

Algorithm 3 Direction-Based Retargeting to the Blender Hand Rig

Require: Refined skeleton $\mathcal{X}^{fit}$; rigged hand mesh $\mathcal{M}$; joint-to-bone mapping table

Ensure: Posed hand mesh $\mathcal{M}^{pose}$

- 1: Clear the previous pose of the hand armature
 - 2: Place $\mathcal{X}^{fit}$ relative to the wrist position of the Blender rig
 - 3: **for** each skeleton segment (j_0, j_1) in parent-to-child order **do**
 - 4: Obtain the corresponding pose bone b from the joint-to-bone mapping
 - 5: Compute target skeleton direction $\mathbf{d}_{j_0, j_1}^{target}$
 - 6: Compute current bone direction $\mathbf{d}_b^{bone}$
 - 7: Normalize both directions to obtain $\mathbf{a}$ and $\mathbf{b}$
 - 8: Compute alignment axis $\mathbf{k} = \mathbf{a} \times \mathbf{b}$
 - 9: Compute rotation angle $\theta = \cos^{-1}(\mathbf{a}^T \mathbf{b})$
 - 10: Construct quaternion $\mathbf{q}$ from $\mathbf{k}$ and θ
 - 11: Update pose-bone matrix around the bone head
 - 12: **end for**
 - 13: Apply armature deformation to obtain $\mathcal{M}^{pose}$
 - 14: **return** $\mathcal{M}^{pose}$
-

3.10 Synthetic multi-view rendering and camera-space skeleton generation

After retargeting, the output is a posed hand mesh $\mathcal{M}^{pose}$. The mesh is observed from multiple virtual camera viewpoints to produce RGB visualizations and camera-space skeleton views.

In the final recognition protocol, RGB images are not used as the main recognition input. They are used for qualitative validation. The same virtual camera setup is used to generate camera-space skeleton views, which provide the training-time multi-view supervision.

The purpose of multi-view generation is to increase viewpoint diversity while preserving the underlying gesture configuration [2]. A static hand gesture may appear significantly different when observed from different camera angles. Some fingers may become partially occluded, the palm orientation may change, and the relative positions of fingertips may appear compressed or expanded in the image plane. By observing the same fitted hand pose from multiple virtual cameras, the system generates several view-dependent observations that share the same gesture label.

Let the rendered image set for one input pose be denoted by

$$\mathcal{R} = \{I_1, I_2, \dots, I_K\},$$

where K is the number of camera views. Each image I_k is generated from the same posed mesh $\mathcal{M}^{pose}$, but under a different camera configuration or visual condition. Therefore, the rendering operation can be represented as

$$I_k = \mathcal{S}(\mathcal{M}^{pose}, \mathbf{C}_k, \Phi_k), \quad (3.30)$$

where $\mathcal{S}(\cdot)$ denotes the rendering process, $\mathbf{C}_k$ denotes the k -th camera configuration, and Φ_k denotes the set of visual rendering parameters such as lighting, material appearance, and background.

3.10.1 Virtual camera placement

The virtual cameras are placed around the posed hand mesh to generate multiple views of the same gesture. The hand center is first estimated from the visible hand objects in the Blender scene. Let $\mathbf{c}_{hand} \in \mathbb{R}^3$ denote the hand center. A camera viewpoint is parameterized by an azimuth angle ϕ_k , a camera radius r , and a height offset h .

The position of the k -th virtual camera is computed as

$$\mathbf{C}_k = \mathbf{c}_{hand} + \begin{bmatrix} r \sin \phi_k \\ -r \cos \phi_k \\ h \end{bmatrix}. \quad (3.31)$$

This formulation places cameras on a horizontal circle around the hand while allowing a fixed vertical offset. The azimuth angle controls the viewing direction around the hand, the radius controls the distance between the camera and the hand, and the height controls whether the hand is viewed from a slightly elevated or lower position.

Each camera is oriented to look at the hand center. The viewing direction of camera k is therefore

$$\mathbf{v}_k = \mathbf{c}_{hand} - \mathbf{C}_k. \quad (3.32)$$

In Blender, this viewing direction is converted into a camera rotation using a look-at transformation. The camera optical axis is aligned with $\mathbf{v}_k$, so that the hand remains centered in the rendered image. This camera placement strategy makes the generated images consistent and avoids views where the hand is outside the frame.

3.10.2 Orthographic rendering

The proposed rendering stage uses an orthographic camera. In orthographic rendering, parallel rays are used to project 3D objects onto the image plane. Unlike perspective rendering, object size does not change with camera depth. This property is useful for synthetic gesture image generation because it reduces unnecessary scale variation caused solely by camera distance.

The orthographic projection of a point in camera coordinates can be written as

$$\mathbf{r}_i = \begin{bmatrix} x_i \\ y_i \end{bmatrix},$$

where (x_i, y_i, z_i) is the 3D coordinate of a point in the camera coordinate system and $\mathbf{r}_i$ is its projected image-plane coordinate before rasterization. The depth coordinate z_i is still used internally for visibility, occlusion, and depth ordering, but it does not affect the projected size of the object.

Using orthographic rendering makes the visual differences between views mainly depend on hand orientation and self-occlusion rather than on perspective scaling. This is consistent with the objective of the synthetic generation stage, which focuses on view-point variation and pose appearance rather than on camera calibration effects.

3.10.3 Rendering configuration

For each posed hand mesh, the renderer generates K images corresponding to K virtual camera views. In the implemented pipeline, the camera views are sampled from a predefined set of azimuth angles. The rendering resolution, camera radius, camera height, and orthographic scale are fixed during a rendering run to ensure consistency across samples.

Table 3.2 summarizes the main rendering parameters used in the pipeline.

Table 3.2: Rendering configuration used for synthetic multi-view image generation.

Parameter	Value / Description
Rendering engine	Blender Cycles
Image resolution	512×512 pixels
Camera type	Orthographic
Camera radius	35.0 Blender units
Camera height	8.0 Blender units
Orthographic scale	28.0
Camera azimuth angles	$-45^\circ, 0^\circ, 30^\circ, 45^\circ, 60^\circ, 90^\circ, 120^\circ, 135^\circ$
Output format	RGB image

The values in Table 3.2 may be adjusted depending on the desired dataset size and rendering cost. However, the important principle is that all views of the same input pose are generated from the same fitted hand mesh. Therefore, differences among the rendered images mainly come from viewpoint and visual rendering conditions, rather than from changes in gesture identity.

3.10.4 Appearance and background variation

In addition to viewpoint diversity, the renderer may vary visual appearance factors such as hand material, skin tone, nail appearance, illumination, and background. These factors are useful for qualitative visualization because they demonstrate how the reconstructed hand pose can be observed under different visual conditions.

The hand material controls the visible surface appearance of the mesh. In the Blender scene, the material can be adjusted through shader parameters such as base color, roughness, specular reflection, and subsurface scattering. These parameters influence the perceived skin color, smoothness, and shading response of the hand. Nail appearance can also be assigned a separate material so that the rendered hand remains visually plausible.

Lighting and background are also important for synthetic image generation. Environment lighting can be provided by an HDRI background, while additional light sources may be added if stronger illumination control is required. Background variation introduces contextual diversity and reduces the risk that rendered examples only represent a single clean studio setup.

Let Φ_k represent the visual rendering configuration of the k -th rendered image. This configuration may include material parameters, illumination settings, and background information. The final rendered set for one pose can therefore be written as

$$\mathcal{R} = \{\mathcal{S}(\mathcal{M}^{pose}, \mathbf{C}_k, \Phi_k)\}_{k=1}^K. \quad (3.33)$$

This formulation emphasizes that the same posed mesh can be rendered multiple times under different visual conditions. The gesture label remains unchanged because the pose configuration is fixed. However, these RGB variations are not the main source of supervision in the final recognition model. The quantitative experiments focus on camera-space skeleton views rather than RGB image recognition.



Figure 3.12: Examples of appearance and background variations.

3.10.5 Camera-space skeleton view generation

Besides RGB rendering, the virtual camera configuration is also used to generate camera-space skeleton observations. In this thesis, camera-space skeleton views can be generated from the skeleton representation used in a given recognition setting. For raw-skeleton experiments, synthetic views are generated from the normalized observed skeleton. For fitted-skeleton experiments, they are generated from the fitted skeleton. This keeps the training views in the same representation domain as the classifier input.

Let the original normalized 3D skeleton be denoted by

$$\mathbf{S} = \{\mathbf{p}_0, \mathbf{p}_1, \dots, \mathbf{p}_{20}\}, \quad \mathbf{p}_i \in \mathbb{R}^3.$$

For the k -th virtual camera, the skeleton is transformed into the coordinate system of that camera:

$$\mathbf{S}^{(k)} = \mathcal{T}_{cam}(\mathbf{S}, \mathbf{C}_k), \quad k = 1, \dots, K, \quad (3.34)$$

where $\mathcal{T}_{cam}(\cdot)$ denotes the camera-space transformation defined by the virtual camera pose.

Each synthetic skeleton view $\mathbf{S}^{(k)}$ represents the same underlying gesture as the orig-

inal skeleton, but from a different viewpoint. Therefore, all synthetic views inherit the same gesture label as the original sample. The complete set of synthetic skeleton views is

$$\mathcal{V} = \{\mathbf{S}^{(1)}, \mathbf{S}^{(2)}, \dots, \mathbf{S}^{(K)}\}. \quad (3.35)$$

This representation is important because it allows the recognition model to learn from multiple virtual viewpoints during training without requiring real multi-camera data. During inference, synthetic views are not required; the model receives only one skeleton.

3.10.6 Generated dataset organization

For each input skeleton sample, the proposed pipeline generates one fitted 3D skeleton, a set of rendered RGB images, and a set of synthetic camera-space skeletons. If the input dataset contains N skeleton samples and each sample is observed from K virtual views, the total number of rendered images is

$$N_{img} = N \times K. \quad (3.36)$$

Similarly, the total number of synthetic camera-space skeleton observations is

$$N_{sk} = N \times K. \quad (3.37)$$

Each generated image and each synthetic skeleton view inherits the gesture label of the corresponding input skeleton. The rendered RGB images are used mainly for visualization, qualitative validation, and exploratory image-based experiments. The synthetic skeleton views are used as training-time supervision in the final recognition model.

The output of the generation stage therefore consists of RGB images, camera-view metadata, fitted skeleton files, and synthetic camera-space skeletons. These outputs provide traceability and make the pipeline easier to inspect and debug.

3.10.7 Summary of the rendering stage

The synthetic rendering stage transforms the posed hand mesh into a multi-view RGB image set and defines the virtual camera configuration used to generate camera-space skeleton observations. By placing virtual cameras around the hand and observing the same pose from multiple viewpoints, the method generates view-diverse samples while preserving gesture identity.

This stage completes the transition from sparse skeleton observation to synthetic multi-view generation. In the final thesis, the rendered RGB images primarily support visualization and qualitative analysis, while the synthetic camera-space skeleton views support viewpoint-robust skeleton recognition.

3.11 Skeleton-based recognition model

The recognition component of the proposed framework is based on 3D hand skeletons. Depending on the experimental setting, the input skeleton may be the original normalized skeleton or the fitted skeleton produced by template-based refinement. Each input sample is represented as an ordered sequence of 21 joints. The model predicts one of six static hand gesture classes: *OK*, *Paper*, *Rock*, *Scissors*, *The-Finger*, and *Thumb*. The *Thumb* class is obtained by merging the original *Thumb-Up* and *Thumb-Down* labels.

Let an input skeleton sample be represented as

$$\mathbf{S} = \{\mathbf{p}_0, \mathbf{p}_1, \dots, \mathbf{p}_{20}\}, \quad \mathbf{p}_j \in \mathbb{R}^3, \quad (3.38)$$

where $\mathbf{p}_j = (x_j, y_j, z_j)$ denotes the coordinate of the j -th hand landmark. The joints are arranged according to the predefined 21-joint hand topology.

Before being fed into the recognition model, the skeleton is normalized. The wrist joint is first translated to the origin:

$$\tilde{\mathbf{p}}_j = \mathbf{p}_j - \mathbf{p}_0. \quad (3.39)$$

The skeleton is then scaled by a reference palm distance. In this thesis, the distance between the wrist joint and the middle metacarpophalangeal joint is used:

$$d = \|\mathbf{p}_9 - \mathbf{p}_0\|_2. \quad (3.40)$$

The normalized joint coordinate is

$$\mathbf{s}_j = \frac{\mathbf{p}_j - \mathbf{p}_0}{d + \epsilon}, \quad (3.41)$$

where ϵ is a small constant used for numerical stability.

The normalized skeleton is written as

$$\mathbf{S} = \{\mathbf{s}_0, \mathbf{s}_1, \dots, \mathbf{s}_{20}\}, \quad \mathbf{s}_j \in \mathbb{R}^3. \quad (3.42)$$

For simplicity, the normalized skeleton is also denoted by $\mathbf{S}$ in the recognition model. A Gated Recurrent Unit (GRU) is used as the skeleton encoder [7]. Although the gestures considered in this thesis are static, the ordered hand skeleton can be interpreted as a structured joint sequence following the hand topology. Processing joints sequentially allows the network to aggregate spatial information from the wrist toward the fingertips and encode the global hand configuration into a compact representation.

The hidden representation at joint position j is computed as

$$\mathbf{h}_j = \text{GRU}(\mathbf{s}_j, \mathbf{h}_{j-1}), \quad j = 0, 1, \dots, 20. \quad (3.43)$$

After all joints have been processed, the final hidden state is used as the skeleton feature representation:

$$\mathbf{f} = \mathbf{h}_{20}. \quad (3.44)$$

The skeleton feature $\mathbf{f}$ is passed to a linear classification head:

$$\mathbf{o} = \Psi(\mathbf{f}), \quad (3.45)$$

where $\Psi(\cdot)$ denotes the classification head and $\mathbf{o} \in \mathbb{R}^C$ is the output logit vector. In the final experimental protocol, $C = 6$.

The predicted class probability is obtained using the softmax function:

$$P(y = c | \mathbf{S}) = \frac{\exp(o_c)}{\sum_{i=1}^C \exp(o_i)}. \quad (3.46)$$

The predicted gesture label is

$$\hat{y} = \arg \max_c P(y = c | \mathbf{S}). \quad (3.47)$$

3.12 Original-anchored multi-view skeleton supervision

Synthetic multi-view skeleton supervision is used to make the model less sensitive to viewpoint changes. For each observed skeleton $\mathbf{S}$, the virtual camera setup generates K camera-space skeleton views:

$$\mathbf{S}^{(1)}, \mathbf{S}^{(2)}, \dots, \mathbf{S}^{(K)}.$$

All synthetic views share the same gesture label as the original skeleton.

The original skeleton and its synthetic views are processed by the same GRU encoder:

$$\mathbf{f}_0 = \Phi_{gru}(\mathbf{S}), \quad (3.48)$$

$$\mathbf{f}_k = \Phi_{gru}(\mathbf{S}^{(k)}), \quad k = 1, \dots, K, \quad (3.49)$$

where $\Phi_{gru}(\cdot)$ denotes the shared skeleton encoder.

Each feature is passed through the same classification head:

$$\mathbf{o}_0 = \Psi(\mathbf{f}_0), \quad \mathbf{o}_k = \Psi(\mathbf{f}_k). \quad (3.50)$$

The classification loss is computed over both the original skeleton and all synthetic views:

$$\mathcal{L}_{cls} = \frac{1}{K+1} \left[CE(\mathbf{o}_0, y) + \sum_{k=1}^K CE(\mathbf{o}_k, y) \right], \quad (3.51)$$

where $CE(\cdot)$ denotes the cross-entropy loss and y denotes the ground-truth gesture label.

To encourage viewpoint-invariant representation learning, an original-anchored consistency loss is introduced. This loss encourages the feature representation of each synthetic view to remain close to the feature representation of the original skeleton. Cosine distance is used:

$$\mathcal{L}_{con} = \frac{1}{K} \sum_{k=1}^K \left(1 - \frac{\mathbf{f}_0^T \mathbf{f}_k}{\|\mathbf{f}_0\|_2 \|\mathbf{f}_k\|_2} \right). \quad (3.52)$$

The final training objective is

$$\mathcal{L} = \mathcal{L}_{cls} + \lambda \mathcal{L}_{con}, \quad (3.53)$$

where λ controls the contribution of the consistency term.

This formulation uses synthetic multi-view information only during training. During inference, the model receives only a single skeleton from the corresponding representation domain:

$$\hat{y} = \arg \max_c P(y = c | \mathbf{S}_{test}). \quad (3.54)$$

Here, $\mathbf{S}_{test}$ denotes either a raw skeleton or a fitted skeleton, depending on the evaluated protocol. Therefore, the method improves viewpoint robustness without requiring multiple synchronized views at test time. Figure 3.13 shows the proposed original-anchored multi-view skeleton supervision architecture.

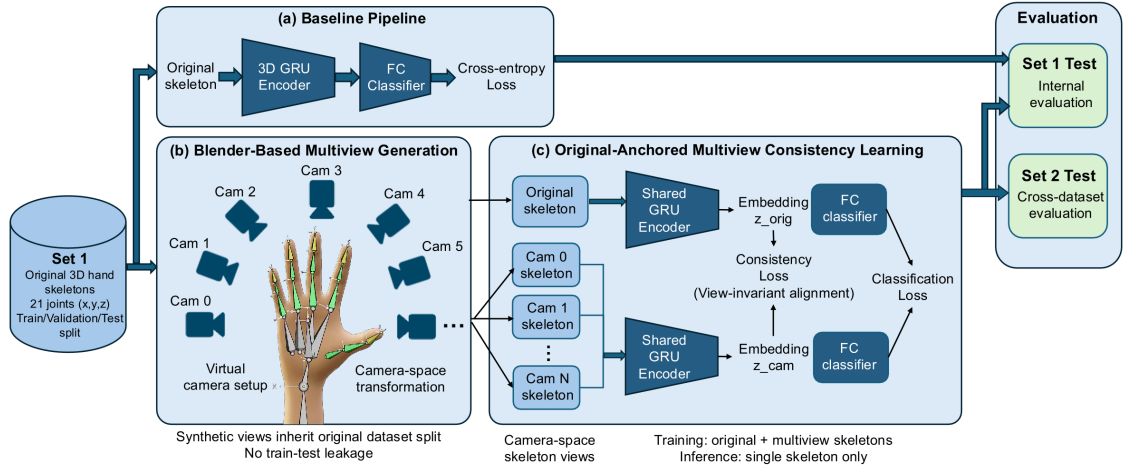


Figure 3.13: Architecture of the proposed original-anchored synthetic multi-view skeleton supervision model. During training, the input skeleton and its synthetic camera-space views are processed by a shared GRU encoder with classification and consistency losses. During inference, only one skeleton from the evaluated representation domain is required.

3.13 Template-based refinement analysis

In addition to the raw skeleton recognition pipeline, this thesis analyzes template-based fitted skeletons. The motivation is that raw skeletons extracted by a hand landmark estimator may contain unstable joints, distorted finger chains, or geometrically implausible configurations. The fitting and refinement stages can produce skeletons that are more regular and more suitable for visualization, retargeting, and synthetic rendering.

However, fitted skeletons should not be treated as a direct replacement for raw skeletons without further analysis. Raw skeletons and fitted skeletons are produced by different processes and may belong to different representation domains. A classifier trained on fitted skeletons may perform well when evaluated on fitted skeletons, but its performance can degrade when tested directly on raw skeletons. Therefore, this thesis evaluates fitted skeletons mainly to study the effect of geometric refinement and the raw–fitted domain mismatch.

Two refinement settings are considered. The first setting is an oracle fitted setting, where the correct gesture template is used during fitting. This setting is useful for estimating the potential upper bound of template-based refinement, but it is not a deployable protocol because it uses label information during fitting. For this reason, oracle fitted results are reported as an analysis rather than as the main performance claim.

The second setting is label-free template selection. In this setting, each input skeleton is fitted to multiple gesture templates, and the final template is selected without using the

ground-truth label. This avoids oracle template selection but turns refinement into a joint template-selection and recognition problem. The label-free experiment is therefore treated as an exploratory diagnostic analysis, not as the main recognition pipeline.

3.14 Overall algorithm

The complete workflow contains two stages: offline generation and skeleton-based recognition. The offline stage reconstructs a plausible 3D hand pose, retargets it to the Blender hand rig, and generates camera-space skeleton views. The recognition stage uses the observed skeleton and its synthetic views during training. At inference time, RGB images and synthetic views are not required; the model receives only one skeleton.

Algorithm 4 summarizes the final training and inference procedure used in the proposed recognition framework.

Algorithm 4 Proposed Synthetic Multi-View Skeleton Supervision Framework

Require: Original skeleton $\mathbf{S}$; observed 2D skeleton $\mathcal{U}$; canonical template $\bar{\mathcal{X}}$; rigged hand mesh $\mathcal{M}$; virtual camera set $\mathcal{C} = C_1, \dots, C_K$; gesture label y

Ensure: Predicted gesture label $\hat{y}$

- 1: Normalize the original skeleton $\mathbf{S}$
 - 2: Fit the canonical template $\bar{\mathcal{X}}$ to $\mathcal{U}$ using global alignment
 - 3: Refine local finger articulation to obtain $\mathcal{X}^{fit}$
 - 4: Retarget $\mathcal{X}^{fit}$ to the Blender hand rig to obtain $\mathcal{M}^{pose}$
 - 5: **for** each virtual camera $C_k \in \mathcal{C}$ **do**
 - 6: Render RGB image I_k for qualitative visualization
 - 7: Generate camera-space skeleton $\mathbf{S}^{(k)}$
 - 8: **end for**
 - 9: Encode observed skeleton: $\mathbf{f}_0 = \Phi_{gru}(\mathbf{S})$
 - 10: **for** each synthetic view $\mathbf{S}^{(k)}$ **do**
 - 11: Encode synthetic view: $\mathbf{f}_k = \Phi_{gru}(\mathbf{S}^{(k)})$
 - 12: **end for**
 - 13: Compute classification loss $\mathcal{L}_{cls}$ over the observed skeleton and synthetic views
 - 14: Compute original-anchored consistency loss $\mathcal{L}_{con}$
 - 15: Optimize $\mathcal{L} = \mathcal{L}_{cls} + \lambda \mathcal{L}_{con}$
 - 16: During inference, predict $\hat{y}$ using one skeleton input from the evaluated domain $\mathbf{S}$
 - 17: **return** $\hat{y}$
-

3.15 Chapter summary

This chapter presented the proposed skeleton-guided framework for synthetic multi-view generation and skeleton-based gesture recognition. The method starts from a 21-joint hand

skeleton and reconstructs a plausible 3D hand pose using a canonical template. Global alignment estimates the coarse pose, while hierarchical refinement adjusts local finger articulation.

The fitted skeleton is transferred to a rigged hand mesh in Blender through direction-based retargeting. The posed mesh is then observed from multiple virtual cameras. RGB renderings are used for qualitative visualization, while camera-space skeleton views are used for multi-view supervision.

The recognition model uses a GRU skeleton encoder. During training, the observed skeleton and its synthetic views are optimized with classification and original-anchored consistency losses. During inference, only one skeleton is required.

Finally, the chapter introduced template-based refinement analysis, including oracle and label-free settings. These settings are used to study the potential and limitations of geometric refinement, not as the main deployable recognition pipeline.

Chapter 4

EXPERIMENTS AND RESULTS

4.1 Experimental setup

This chapter presents the experimental evaluation of the proposed skeleton-guided synthetic multi-view framework for in-the-wild static hand gesture recognition. The main objective is to evaluate whether synthetic multi-view skeleton supervision improves view-point robustness and cross-dataset generalization.

Different from an RGB-centered recognition setting, the final quantitative experiments in this thesis focus on skeleton-based recognition. Rendered RGB images are used mainly for qualitative visualization of the reconstruction and rendering pipeline. The recognition model receives 3D hand skeletons as input, while synthetic camera-space skeleton views are used during training to provide multi-view supervision.

The experiments are designed to evaluate three main aspects of the proposed framework. First, a raw skeleton baseline is established as the main deployable baseline. Second, synthetic camera-space skeleton views are evaluated to determine whether training-time multi-view supervision improves external generalization. Third, geometry-refined skeletons are analyzed as diagnostic settings to study the potential and limitations of template-based refinement.

All recognition experiments are conducted on six static hand gesture classes:

OK, Paper, Rock, Scissors, The-Finger, Thumb.

The original *Thumb-Up* and *Thumb-Down* labels are merged into a single *Thumb* class during evaluation. This merging is adopted because both labels represent the same dominant hand-shape category and may be difficult to separate reliably under viewpoint variation.

CanonicalSet is used as the source dataset for training, validation, internal testing, and synthetic multi-view generation. HandinWildSet is used exclusively as an external hand-in-the-wild test dataset. It is not used for training, validation, checkpoint selection, or hyperparameter tuning. This protocol evaluates whether the learned skeleton representation can generalize beyond the source dataset.

4.2 Dataset description and preparation

4.2.1 Dataset description

The experimental protocol uses two hand gesture datasets: CanonicalSet and HandinWildSet. CanonicalSet is used as the main development dataset, while HandinWildSet is used as an external evaluation dataset. Both datasets originally contain RGB images of static hand gestures collected under hand-in-the-wild conditions. Compared with controlled laboratory datasets, these images include variations in camera viewpoint, illumination, background clutter, hand appearance, scale, and acquisition environment.

The original gesture categories include *OK*, *Paper*, *Rock*, *Scissors*, *The-Finger*, *Thumb-Up*, and *Thumb-Down*. In this thesis, *Thumb-Up* and *Thumb-Down* are merged into one *Thumb* class during evaluation. As a result, the final recognition protocol contains six classes.

The final six-class dataset statistics are summarized in Tables 4.1, 4.2, and 4.3. CanonicalSet contains 6,547 skeleton samples and is split into training, validation, and internal testing subsets with an approximate ratio of 70%, 15%, and 15%, respectively. HandinWildSet contains 605 samples and is used only for external testing.

Table 4.1: Class distribution of the CanonicalSet dataset.

Class	Train	Validation	Test	Total
OK	758	162	163	1083
Paper	840	180	180	1200
Rock	227	49	49	325
Scissors	989	212	212	1413
The-Finger	526	113	112	751
Thumb	1242	266	267	1775
Total	4582	982	983	6547

Table 4.2: Class distribution of the HandinWildSet external test dataset.

Class	Number of samples
OK	106
Paper	123
Rock	109
Scissors	42
The-Finger	103
Thumb	122
Total	605

Table 4.3: Number of synthetic camera-space skeleton views generated from CanonicalSet.

Split	Original skeletons	Views per sample	Synthetic views
Train	4582	8	36656
Validation	982	8	7856
Test	983	8	7864
Total	6547	8	52376

RGB images are converted into 21-joint 3D hand skeletons using a hand landmark extraction pipeline. The skeleton representation is used as the input for all quantitative recognition experiments. The RGB images generated by the proposed Blender pipeline are retained for qualitative visualization and reconstruction analysis, but they are not used as the final recognition modality.

Table 4.4: Summary of datasets and their roles in the experimental protocol.

Dataset	Role	Usage
CanonicalSet	Source dataset	Training, validation, internal testing, synthetic view generation
HandinWildSet	External dataset	External testing only

Figure 4.1 shows representative RGB samples from the datasets. The examples illustrate the diversity of viewpoint, background, lighting condition, and hand appearance, which makes cross-dataset hand gesture recognition challenging.

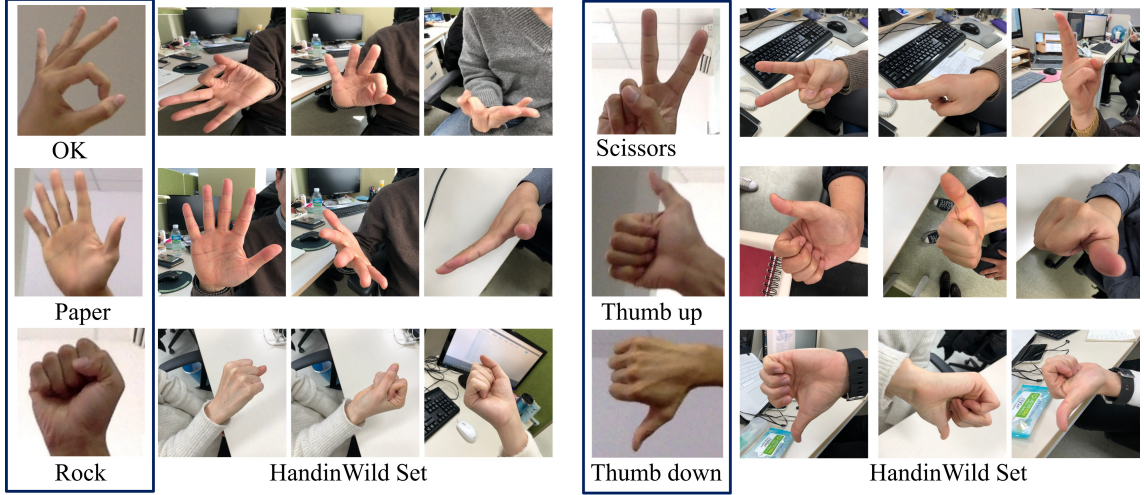


Figure 4.1: Representative RGB samples from the CanonicalSet and HandinWildSet hand gesture datasets. The datasets contain variations in viewpoint, illumination, background, hand appearance, and camera distance.

4.2.2 Skeleton preprocessing

Each skeleton sample is represented as

$$\mathbf{S} = \{\mathbf{p}_0, \mathbf{p}_1, \dots, \mathbf{p}_{20}\}, \quad \mathbf{p}_i \in \mathbb{R}^3. \quad (4.1)$$

The joint topology follows the 21-joint hand skeleton convention described in Chapter 3. Before recognition, each skeleton is normalized by translating the wrist joint to the origin and scaling the hand according to the distance between the wrist and the middle metacarpophalangeal joint. This normalization reduces the influence of global position and hand scale while preserving relative finger configuration.

Two types of skeleton representations are considered. The first type is the raw skeleton, which is directly obtained from the skeleton extraction pipeline. This representation corresponds to the practical deployment setting because the model receives detector outputs directly. The second type is the fitted skeleton, which is obtained by template-based fitting and hierarchical refinement. Fitted skeletons are geometrically more plausible and useful for visualization, but they may differ from raw skeletons in distribution.

The distinction between raw and fitted skeletons is essential. A model trained on fitted skeletons may not generalize well to raw skeletons because the two representations belong to different domains. Therefore, the experiments explicitly evaluate raw-domain performance, fitted-domain performance, oracle fitted evaluation, and direct fitted-to-raw generalization.

4.2.3 Synthetic multi-view generation

For multi-view experiments, each skeleton sample is augmented using eight synthetic camera-space views generated from the Blender-based virtual camera setup. The camera azimuth angles are

$$\Phi = \{-45^\circ, 0^\circ, 30^\circ, 45^\circ, 60^\circ, 90^\circ, 120^\circ, 135^\circ\}. \quad (4.2)$$

These views preserve the original gesture label while introducing controlled viewpoint variation.

Figure 4.2 illustrates the virtual camera layout used to generate the synthetic camera-space skeleton observations.

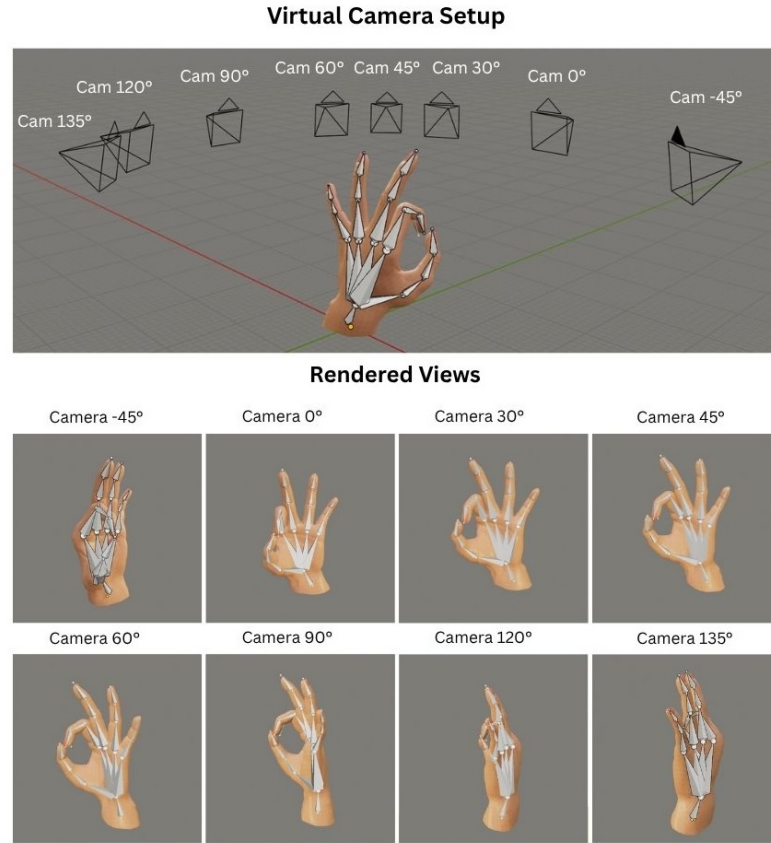


Figure 4.2: Virtual camera layout used for generating eight synthetic multi-view observations of each hand pose.

Synthetic views are generated only after the original dataset split has been fixed. Therefore, all synthetic views generated from a training sample remain in the training set, and all synthetic views generated from validation or test samples remain in their corresponding subsets. This split-preserving generation strategy prevents train-test leakage

and ensures that no synthetic view derived from a test sample is used during training.

4.2.4 Dataset usage protocol

The dataset usage protocol is illustrated in Figure 4.3. CanonicalSet is used for model development and synthetic view generation, while HandinWildSet is reserved exclusively for external testing.

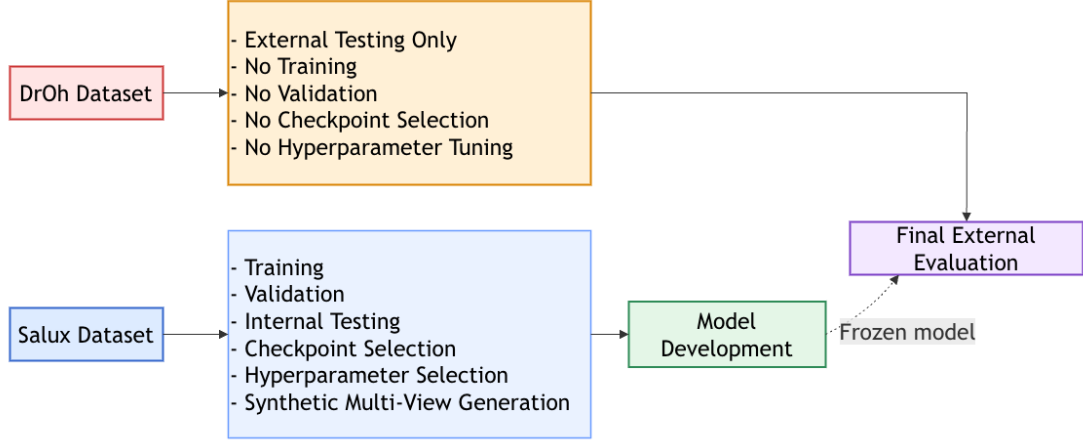


Figure 4.3: Dataset usage and evaluation protocol. CanonicalSet is used for model development, while HandinWildSet is reserved exclusively for external testing.

This protocol allows the experiments to evaluate both internal recognition performance and cross-dataset generalization. Performance on CanonicalSet measures recognition ability within the source dataset, while performance on HandinWildSet measures robustness under different data conditions.

4.3 Implementation details

The recognition model is implemented using PyTorch. The skeleton encoder is a one-layer Gated Recurrent Unit (GRU). Although the gestures are static, the ordered 21-joint skeleton is treated as a structured joint sequence. The GRU processes the joint sequence and outputs a compact feature representation for gesture classification.

The main model configuration is summarized in Table 4.5.

Table 4.5: Main model and training configuration.

Parameter	Value
Model	GRU
Number of GRU layers	1
Hidden size	128
Input shape	21×3
Classifier	Linear 6-class head
Optimizer	Adam
Learning rate	1×10^{-3}
Batch size	64
Maximum epochs	150
Early stopping patience	20
Random seeds	5 independent seeds
Number of synthetic views	8
Main consistency weight λ	0.3

For synthetic multi-view consistency learning, the training loss is

$$\mathcal{L} = \mathcal{L}_{cls} + \lambda \mathcal{L}_{con}. \quad (4.3)$$

Here, $\mathcal{L}_{cls}$ is the average cross-entropy loss over the input skeleton and its synthetic views, while $\mathcal{L}_{con}$ is the cosine consistency loss between the input skeleton embedding and the synthetic-view embeddings. The main controlled setting uses $\lambda = 0.3$. For the main quantitative results, each method is trained with five independent random seeds, and the results are reported as mean and standard deviation.

The RGB-based branch was investigated during development using a ResNet-18 backbone. However, preliminary RGB-based results were not stable because of the visual gap between synthetic rendered images and real hand-in-the-wild data. Therefore, RGB-based recognition is not used as the main quantitative protocol in this thesis. Rendered RGB images are retained for qualitative visualization and analysis of the reconstruction pipeline.

4.4 Evaluation metrics

The main evaluation metric is classification accuracy:

$$Accuracy = \frac{N_{correct}}{N_{total}}, \quad (4.4)$$

where $N_{correct}$ denotes the number of correctly classified samples and N_{total} denotes the total number of test samples. For the main quantitative comparison, accuracy is com-

puted separately for each independent training seed. The final reported result is the mean accuracy and standard deviation over five seeds.

When class-level results are available, precision, recall, and F1-score are also reported. For each gesture category, precision is defined as

$$Precision = \frac{TP}{TP + FP}, \quad (4.5)$$

where TP denotes true positives and FP denotes false positives. Recall is defined as

$$Recall = \frac{TP}{TP + FN}, \quad (4.6)$$

where FN denotes false negatives. The F1-score is computed as

$$F1 = 2 \cdot \frac{Precision \times Recall}{Precision + Recall}. \quad (4.7)$$

Confusion matrices are used to analyze class-specific errors. They are especially useful for identifying pairs of gestures that are difficult to distinguish, such as gestures with similar fingertip arrangements or gestures affected by strong viewpoint changes.

4.5 Experimental protocols

This thesis evaluates four main protocols.

The first protocol is the raw skeleton baseline. The model is trained on CanonicalSet raw skeletons without synthetic multi-view supervision. This protocol is the main deployable baseline.

The second protocol is raw skeleton training with Blender-8 multi-view cross-entropy supervision. The model is trained using the original raw skeletons and eight synthetic camera-space skeleton views. This protocol evaluates the effect of synthetic viewpoint diversity without the consistency term.

The third protocol is raw skeleton training with Blender-8 multi-view consistency. The model uses the same synthetic views as the second protocol, but adds original-anchored consistency learning. This is the final deployable model.

The fourth protocol is geometry-refined skeleton training with Blender-8 multi-view supervision. This setting is used as a diagnostic upper-bound analysis because it evaluates the potential of geometrically refined skeletons. It is not treated as the main deployable

protocol.

Table 4.6: Main experimental protocols.

Protocol	Training input	Multi-view	Main purpose
Raw baseline	Raw skeletons	No	Deployable base-line
Raw + Blender-8 MV CE-only	Raw skeletons + 8 views	Yes	Effect of synthetic multi-view
Raw + Blender-8 MV + Consistency	Raw skeletons + 8 views	Yes	Final deployable model
Geometry-refined + Blender-8 MV	Refined skeletons + 8 views	Yes	Upper-bound diagnostic

4.6 Quantitative results

This section reports the main quantitative results on HandinWildSet. The results are reported as mean accuracy and standard deviation over five independent training seeds. For each seed, accuracy is computed as the number of correctly classified samples divided by the total number of test samples.

Table 4.7 summarizes the main results.

Table 4.7: Summary of main experimental results on HandinWildSet.

Method	Training input	HandinWildSet accuracy (%)
Raw baseline	Raw skeletons	80.86 ± 2.78
Raw + Blender-8 MV CE-only	Raw skeletons + 8 views	84.86 ± 1.35
Raw + Blender-8 MV + Consistency	Raw skeletons + 8 views	86.88 ± 1.40
Geometry-refined + Blender-8 MV	Refined skeletons + 8 views	89.95 ± 3.79

The raw baseline achieves $80.86 \pm 2.78\%$ accuracy on HandinWildSet. When eight synthetic camera-space skeleton views are added with cross-entropy supervision, the accuracy increases to $84.86 \pm 1.35\%$. This result shows that synthetic multi-view training provides useful viewpoint diversity even without an explicit consistency loss.

With original-anchored consistency learning, the final deployable model reaches $86.88 \pm 1.40\%$. Compared with the raw baseline, this corresponds to an improvement of 6.02 percentage points. This result supports the main hypothesis of the thesis: synthetic camera-

space skeleton views can improve external hand-in-the-wild recognition while preserving single-skeleton inference.

The geometry-refined setting achieves the highest accuracy, reaching $89.95 \pm 3.79\%$. This suggests that more regular geometric skeletons can further improve recognition. However, this setting is treated as an upper-bound diagnostic result rather than the main deployable protocol, because reliable automatic refinement and template selection are still required for practical inference.

Overall, the results show two main findings. First, synthetic multi-view supervision improves the deployable raw-skeleton setting from $80.86 \pm 2.78\%$ to $86.88 \pm 1.40\%$. Second, geometry-refined skeletons show higher potential performance, but they should be interpreted as a diagnostic upper-bound setting rather than the final deployable model.

4.6.1 Confusion matrix analysis

Confusion matrices are used to analyze class-level recognition errors. While Table 4.7 reports the mean and standard deviation over five seeds, the confusion matrix provides a representative class-level view of the model behavior.

Figure 4.4 compares the raw skeleton baseline and the raw multi-view model on HandinWildSet. The result shows that synthetic multi-view supervision improves the overall recognition behavior, but some errors remain between gestures with similar finger configurations. These errors are especially visible when fingers are partially occluded, when fingertips are noisy, or when different gestures produce similar projected skeleton structures.

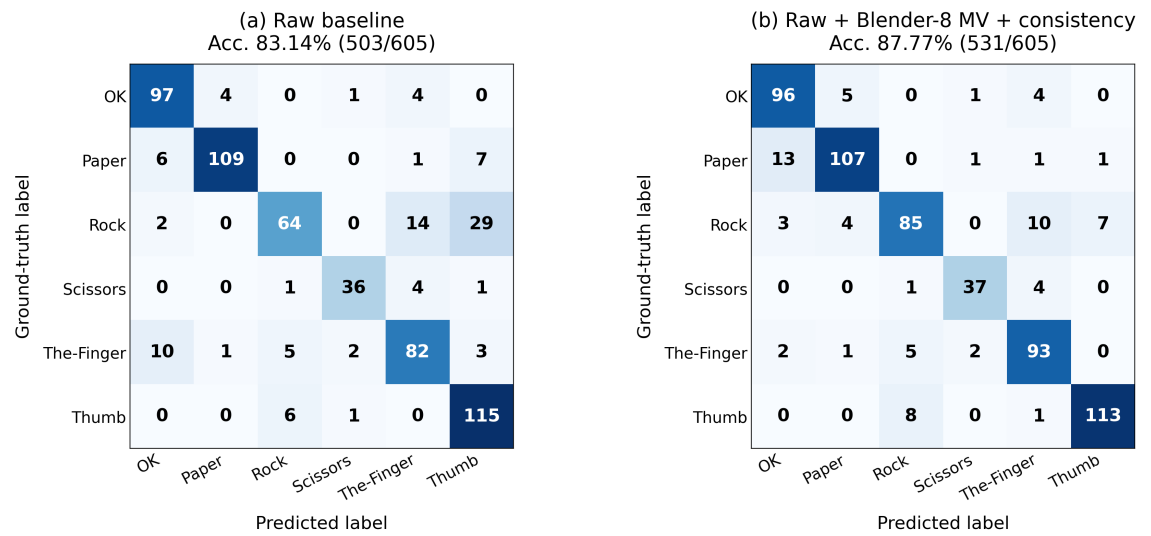


Figure 4.4: Confusion matrices of the raw skeleton baseline and the raw multi-view model on HandinWildSet. Values are normalized by ground-truth class.

To better understand the remaining errors, Figure 4.5 shows representative failure cases from HandinWildSet. These examples illustrate common sources of confusion in hand-in-the-wild gesture recognition.



Figure 4.5: Representative failure cases on HandinWildSet. Errors mainly occur when gestures have similar folded-finger structures, open-palm ambiguity, partial finger visibility, or unclear thumb visibility.

The first example shows a *Rock* gesture predicted as *The-Finger*. This error occurs because the folded fingers create a similar skeleton structure. The second example shows an *OK* gesture predicted as *Paper*, mainly due to open-palm ambiguity and unclear fingertip contact. The third example shows *The-Finger* predicted as *Scissors*, where partial finger visibility changes the apparent finger configuration. The fourth example shows *Thumb* predicted as *Rock*, because the thumb is not clearly visible from the camera viewpoint.

Overall, the confusion matrix and failure examples show that synthetic multi-view training improves viewpoint robustness, but the model can still fail when hand landmarks are noisy, fingers are occluded, or different gestures become geometrically similar under certain viewpoints.

4.7 Qualitative results

Besides quantitative evaluation, qualitative analysis is performed to visualize each stage of the proposed pipeline. Unlike the quantitative experiments, which evaluate recognition accuracy, the qualitative analysis focuses on demonstrating the geometric validity of the reconstructed hand pose, the effectiveness of template fitting, and the quality of the rendered synthetic views.

4.7.1 Template fitting

Figure 4.6 illustrates representative examples of the initial template fitting process. The canonical hand template is first globally aligned with the observed hand skeleton using weak-perspective transformation. Although only a coarse alignment is obtained at this stage, the template already follows the overall orientation and scale of the target hand.

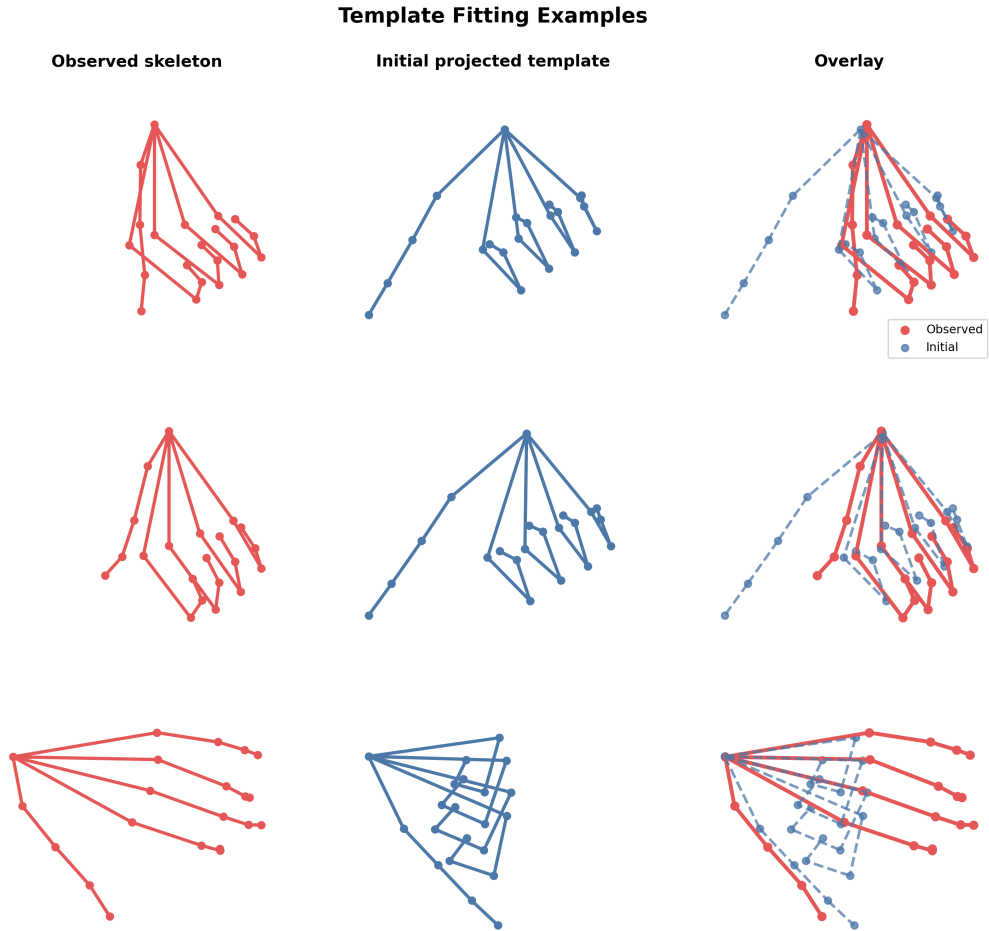


Figure 4.6: Examples of global template alignment between the observed skeleton and the projected canonical template.

The initial alignment mainly estimates the global pose of the hand and provides an appropriate initialization for the subsequent refinement stage. Since individual finger articulations are not yet optimized, noticeable discrepancies remain around the fingertips and highly articulated finger joints.

4.7.2 Hierarchical refinement

Figure 4.7 compares the projected hand skeleton before and after hierarchical refinement.

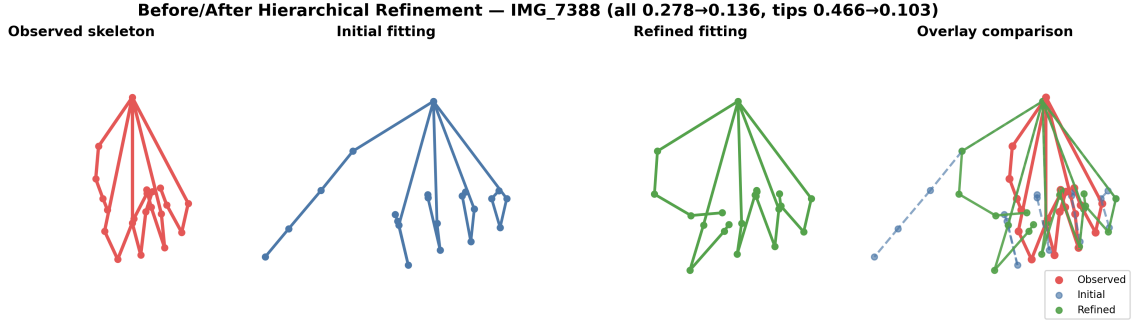


Figure 4.7: Comparison between the initial fitting result and the refined skeleton projection. Hierarchical refinement improves local finger articulation while preserving the hand topology.

After hierarchical refinement, local finger orientations become more consistent with the target gesture. The refinement is performed sequentially from the palm toward each fingertip, allowing each finger chain to be adjusted independently while preserving anatomical constraints inherited from the parent joints.

Compared with the initial fitting stage, the refined skeleton exhibits better alignment at distal finger joints and produces smoother finger configurations. This refinement is important because it directly influences the subsequent rig retargeting and rendering quality.

4.7.3 Rig retargeting

After skeleton refinement, the fitted hand pose is transferred to the Blender hand rig through bone direction alignment.

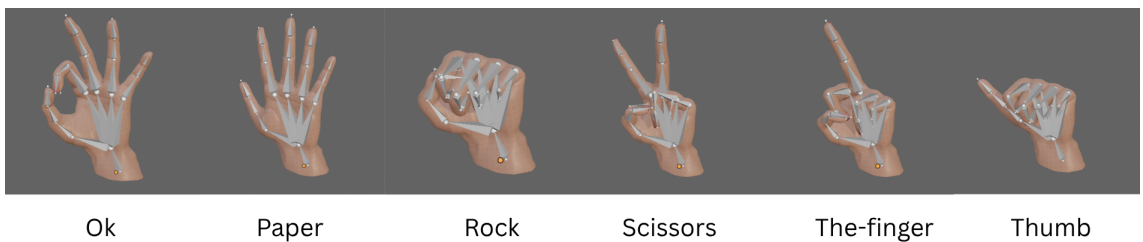


Figure 4.8: Examples of retargeting fitted skeletons to the Blender hand rig and generating posed hand meshes.

As illustrated in Figure 4.8, the generated hand mesh reproduces the target gesture while maintaining realistic hand geometry. Since the hand mesh is controlled through an articulated armature rather than direct vertex manipulation, the resulting deformation remains smooth and anatomically plausible.

The retargeted hand mesh serves as the foundation for subsequent synthetic image generation.

4.7.4 Synthetic multi-view rendering

Once the rigged hand mesh has been generated, RGB images are rendered from multiple predefined virtual viewpoints.



Figure 4.9: Examples of synthetic multi-view rendering generated from one reconstructed hand pose.

Figure 4.9 demonstrates that all rendered images correspond to the same reconstructed hand pose while exhibiting different viewing angles. Because viewpoint changes are performed directly in the three-dimensional scene, gesture semantics remain unchanged throughout the rendering process.

Compared with conventional two-dimensional image augmentation, the proposed rendering strategy produces more realistic viewpoint changes, including self-occlusion and viewpoint-dependent hand appearance.

These rendered images are mainly used for qualitative visualization in the final framework. During the development stage of the project, they were also investigated for RGB-based recognition experiments.

4.7.5 Recognition examples

Figure 4.10 presents representative recognition examples obtained by the proposed skeleton-based recognition framework.

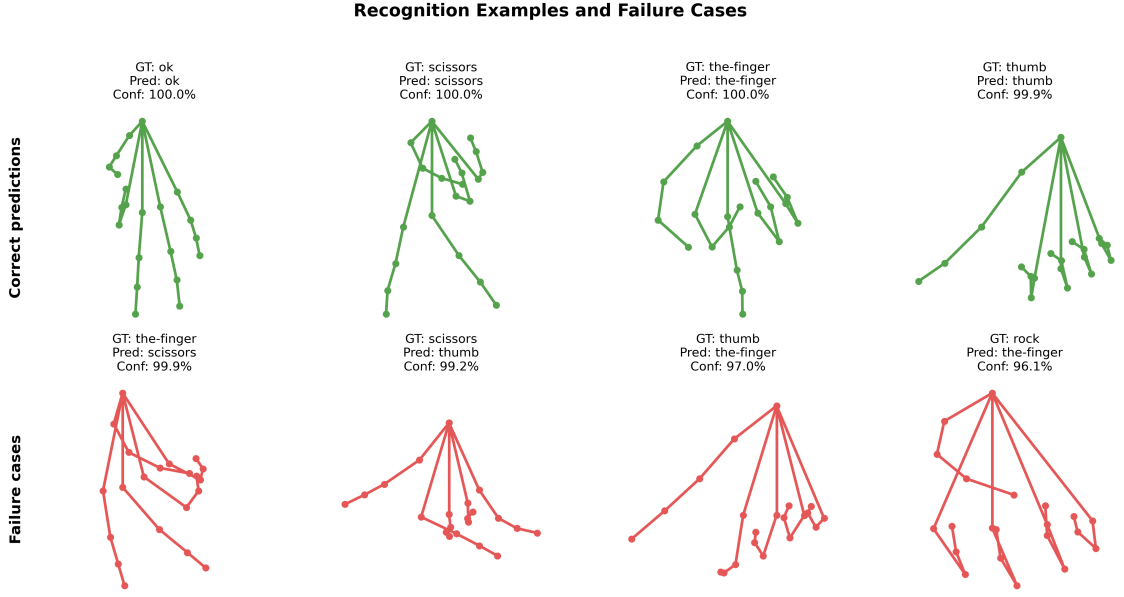


Figure 4.10: Representative examples of successful recognition and failure cases. Errors mainly occur when gestures have similar finger configurations or when raw skeleton estimation is unstable.

Most gesture categories are correctly classified under different viewpoints. Representative failure cases mainly occur when multiple gestures exhibit highly similar finger configurations or when raw skeleton estimation contains noticeable joint localization errors.

For example, gestures involving partially folded fingers may occasionally be confused because their geometric structures become similar after projection into certain viewpoints. In addition, inaccuracies in fingertip localization may propagate through the feature extraction process and reduce classification confidence.

Nevertheless, the overall qualitative results demonstrate that the proposed reconstruction pipeline produces anatomically plausible hand poses and that the generated synthetic views preserve the semantic consistency of the original gesture.

4.8 Discussion

The experimental findings indicate that synthetic camera-space skeleton views effectively enhance skeleton-based hand gesture recognition performance under external in-the-wild hand testing. The raw baseline achieves $80.86 \pm 2.78\%$ on HandinWildSet. After incorporating eight synthetic views with cross-entropy supervision, the accuracy rises to $84.86 \pm 1.35\%$. Further applying original-anchored consistency learning yields a final

deployable model with an accuracy of $86.88 \pm 1.40\%$.

This improvement shows that the generated synthetic views provide useful viewpoint diversity during training. Importantly, this gain is obtained without requiring multi-view input during inference. The deployed model still receives only one observed skeleton.

The geometry-refined result reaches $89.95 \pm 3.79\%$, which is higher than the raw deployable setting. This suggests that more regular geometric skeletons can improve recognition. However, this setting should be interpreted as a diagnostic upper-bound analysis. In the current framework, reliable automatic refinement and template selection remain open problems.

The qualitative results support the quantitative findings. Global template fitting provides a coarse alignment between the observed skeleton and the canonical template. Hierarchical refinement improves local finger articulation. Rig retargeting transfers the refined skeleton to a Blender hand mesh, and virtual cameras generate viewpoint-diverse observations.

Overall, the experiments support two main conclusions. First, synthetic multi-view skeleton supervision improves the raw deployable model under external testing. Second, geometry refinement is promising, but it should be treated as a diagnostic analysis until a robust automatic refinement strategy is available.

4.9 Limitations

Despite the improved results, several limitations remain.

First, the model still makes errors when gestures have similar finger configurations, especially under self-occlusion or difficult viewpoints. Small landmark errors around fingertips can also affect classification.

Second, the proposed generation pipeline relies on a fixed hand template and predefined virtual camera views. This design is useful for controlled analysis, but it does not fully cover all hand shapes, camera positions, and real-world pose variations.

Third, the geometry-refined setting is not yet a fully deployable protocol. Although it achieves strong performance, robust automatic refinement and template selection are still required before fitted-skeleton recognition can be used reliably at inference time.

Finally, the experiments focus on static hand gestures. Dynamic gestures, temporal motion patterns, and continuous gesture segmentation are outside the scope of this thesis.

4.10 Chapter summary

This chapter presented the experimental evaluation of the proposed framework. CanonicalSet was used for training, validation, internal testing, and synthetic view generation, while HandinWildSet was used only for external testing.

The main results show that synthetic multi-view training improves HandinWildSet accuracy from $80.86 \pm 2.78\%$ to $84.86 \pm 1.35\%$ with cross-entropy supervision, and to $86.88 \pm 1.40\%$ with original-anchored consistency learning. This confirms that synthetic camera-space skeleton views improve external recognition while preserving single-skeleton inference.

The geometry-refined setting reaches $89.95 \pm 3.79\%$, showing the potential of geometric refinement. However, this setting is treated as a diagnostic upper-bound analysis rather than the main deployable result.

Qualitative results show that the proposed pipeline can perform template fitting, hierarchical refinement, rig retargeting, and synthetic multi-view generation in a geometrically meaningful way. Overall, the experiments validate the main raw-skeleton multi-view supervision pipeline and clarify the current limitations of geometry-refined recognition.

CONCLUSIONS AND FUTURE WORK

Conclusions

This thesis presented a geometry-driven framework for improving viewpoint robustness in static hand gesture recognition using 3D skeleton representations. Instead of relying only on conventional two-dimensional augmentation, the proposed method generates synthetic camera-space skeleton views from observed hand skeletons and uses them as training-time supervision.

The proposed framework consists of three main parts. First, an observed skeleton is fitted to a canonical 3D hand template using global alignment and hierarchical refinement. Second, the fitted skeleton is retargeted to a rigged Blender hand model and observed from multiple virtual camera viewpoints. Third, the generated camera-space skeleton views are used to train a skeleton-based gesture classifier. The rendered RGB images are used mainly for qualitative visualization, while the final recognition model remains skeleton-based.

For recognition, this thesis uses a GRU-based skeleton classifier. During training, each observed skeleton is paired with synthetic camera-space skeleton views. Cross-entropy supervision and original-anchored consistency learning are used to encourage more viewpoint-stable representations. During inference, the model requires only one skeleton input, so no multi-camera setup is needed at test time.

The experimental results show that synthetic multi-view training improves external HandinWildSet performance. The raw skeleton baseline achieves $80.86 \pm 2.78\%$. With synthetic multi-view cross-entropy supervision, the accuracy increases to $84.86 \pm 1.35\%$. With original-anchored consistency learning, the final deployable model reaches $86.88 \pm 1.40\%$. This corresponds to an improvement of 6.02 percentage points over the raw baseline while preserving single-skeleton inference.

The geometry-refined setting achieves $89.95 \pm 3.79\%$, which shows the potential of

more regular skeleton representations. However, this setting is treated as a diagnostic upper-bound result rather than the main deployable protocol. Reliable automatic refinement and template selection are still required before geometry-refined recognition can be used robustly at inference time.

The qualitative results further confirm that the proposed pipeline can produce meaningful geometric transformations. Global template fitting provides a coarse alignment, hierarchical refinement improves local finger articulation, rig retargeting transfers the skeleton to a posed Blender hand mesh, and virtual cameras generate viewpoint-diverse observations.

Overall, the thesis demonstrates that synthetic camera-space skeleton views are effective for improving hand-in-the-wild gesture recognition. The main contribution is a training-time multi-view supervision strategy that improves external recognition performance without increasing inference complexity.

Future work

Future work can focus on three directions.

First, more robust automatic geometry refinement should be investigated. The current geometry-refined setting shows promising results, but it is still treated as a diagnostic setting. A reliable automatic refinement strategy is needed before this setting can be used in practical inference.

Second, stronger skeleton recognition architectures can be explored. Graph neural networks or transformer-based models may better capture spatial relationships among hand joints than the current GRU baseline.

Third, the framework should be evaluated on additional hand gesture datasets. More cross-dataset experiments would provide a clearer assessment of viewpoint robustness and real-world generalization.

Bibliography

- [1] J. Shin, A. S. M. Miah, M. H. Kabir, M. A. Rahim, and A. Al Shiam. A methodological and structural review of hand gesture recognition across diverse data modalities. *IEEE Access*, 12:142606–142639, 2024.
- [2] Gul Varol, Ivan Laptev, Cordelia Schmid, and Andrew Zisserman. Synthetic humans for action recognition from unseen viewpoints. *International Journal of Computer Vision*, 129:2264–2287, 2021.
- [3] Fan Zhang, Valentin Bazarevsky, Andrey Vakunov, Andrei Tkachenka, George Sung, Chuo-Ling Chang, and Matthias Grundmann. Mediapipe hands: On-device real-time hand tracking. In *CVPR Workshops*, 2020.
- [4] Jun Liu, Amir Shahroudy Wang, et al. Deep learning for skeleton-based human action recognition: A survey. *Pattern Recognition*, 107, 2020.
- [5] Huong-Giang Doan. New blender-based augmentation method for hand gesture recognition. *Indonesian Journal of Electrical Engineering and Computer Science*, 30(2):796–806, 2023.
- [6] Blender Online Community. Blender - a 3d modelling and rendering package, 2024.
- [7] Kyunghyun Cho, Bart van Merriënboer, Caglar Gulcehre, et al. Learning phrase representations using rnn encoder-decoder for statistical machine translation. In *EMNLP*, pages 1724–1734, 2014.
- [8] Chunyu Bian, Wenhao Feng, and Fei Meng. View-invariant skeleton-based action recognition via global-local contrastive learning. *arXiv preprint arXiv:2209.11634*, 2022.
- [9] Yong He, Zhen Liang, Shuai He, Yifan Wang, and Ming Yin. Viewpoint guided multi-stream neural network for skeleton action recognition. *Multimedia Tools and Applications*, 83, 2023.

- [10] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *CVPR*, pages 770–778, 2016.
- [11] Takuma Ohkawa, Ryosuke Furuta, and Yoichi Sato. Efficient annotation and learning for 3d hand pose estimation: A survey. *International Journal of Computer Vision*, 131:3193–3206, 2023.
- [12] Andreas Aristidou and Joan Lasenby. Inverse kinematics: A review of existing techniques and introduction of a new fast iterative solver. *Computer Graphics Forum*, 28(1):243–260, 2009.
- [13] Chao Guo and Di Wu. Canonical correlation analysis based multi-view classification: An overview. *arXiv preprint arXiv:1907.01693*, 2021.
- [14] Huu-Nhan Tran, Huu-Quoc Nguyen, Hai-Giang Doan, Thi-Hanh Tran, Thi-Lan Le, and Hai Vu. Pairwise-covariance multi-view discriminant analysis for robust cross-view human action recognition. *IEEE Access*, 9:130410–130422, 2021.
- [15] Prannay Khosla, Piotr Teterwak, Chen Wang, et al. Supervised contrastive learning. In *NeurIPS*, 2020.
- [16] Javier Romero, Dimitrios Tzionas, and Michael J. Black. Embodied hands: Modeling and capturing hands and bodies together. *ACM Transactions on Graphics*, 36(6), 2017.